

Ch 1: Introduction to Basic Data Structure and Algorithm Design

Data Structure and Algorithm Design

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal
Former Guest Faculty, Darmstadt University of Applied Sciences, Germany

CamScanner



Overview

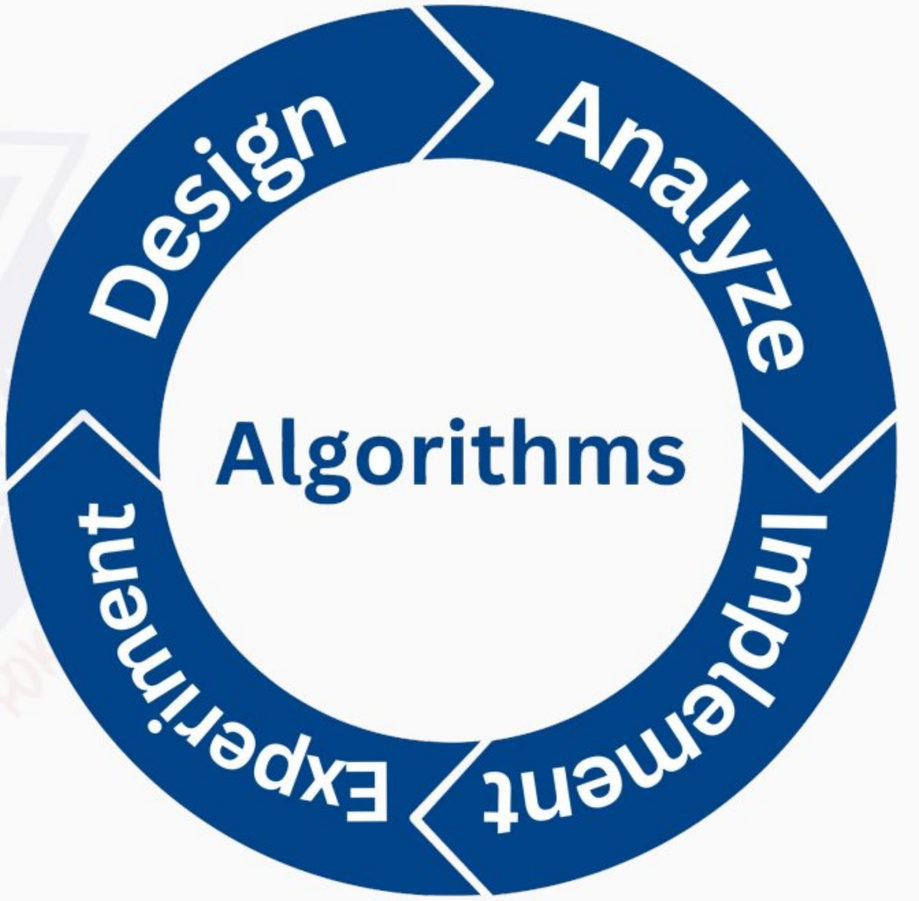
Ch 1: Introduction to Basic Data Structure and Algorithm Design

- Basic Data Structures
  - Arrays, Stack, Queue, Linked list, Tree and Graph
- Algorithm Design Techniques
  - Divide and Conquer
  - Greedy Algorithm
  - Backtracking Algorithms
  - Branch & Bound
  - Randomized Algorithms
- Algorithm Analysis
  - Time and Space Complexity
  - Asymptotic notations: Big O, Big Theta and Big Omega

CamScanner

Introduction to Algorithm Design

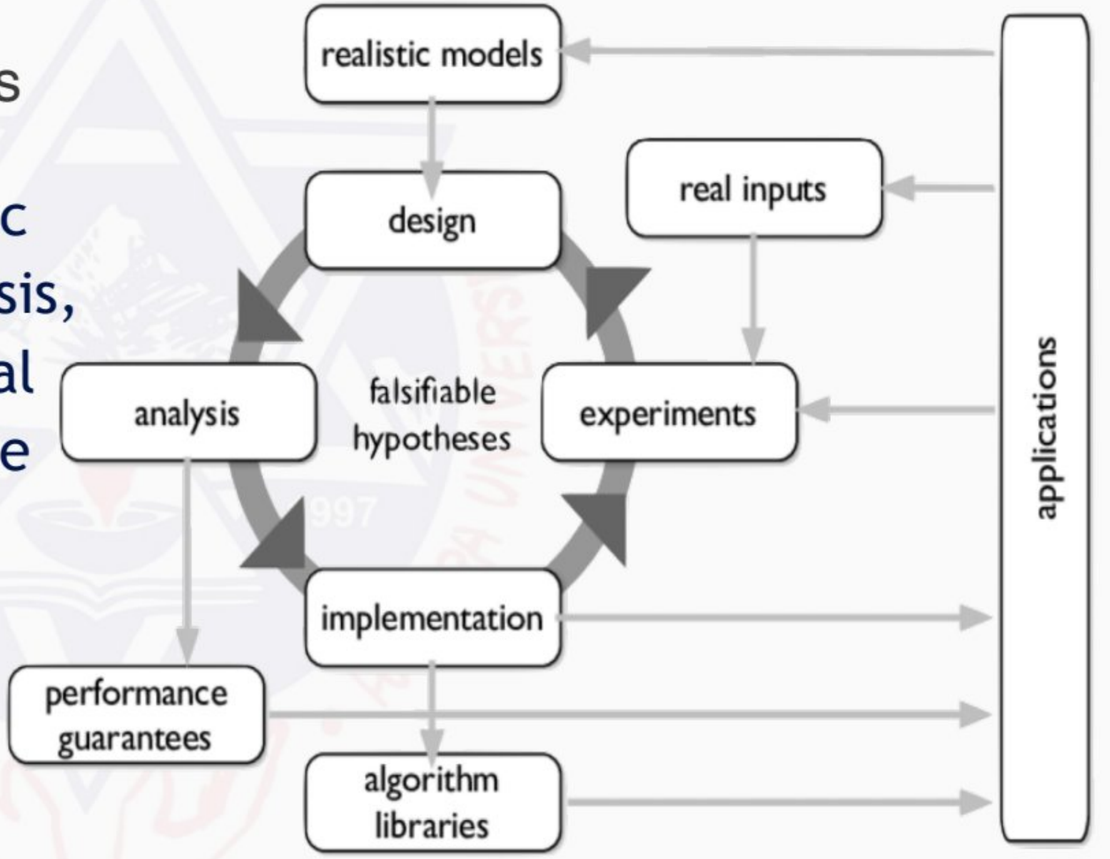
- An algorithm design technique is an approach or mathematical method for creating/constructing efficient algorithms to the problems.
- Multiple algorithms can solve a problem, but not all can solve it efficiently.
- An algorithm created with the right design technique based on the nature of the problem can solve the problem much more efficiently



CamScanner

Algorithm Engineering

- Deals with the systematic development of efficient algorithms
- It is a methodology for algorithmic research that views design, analysis, implementation, and experimental evaluation of algorithms as a cycle driving algorithmic research.
- It bridges the gap between algorithm theory and practical applications of algorithms.



CamScanner

Algorithm Design Techniques

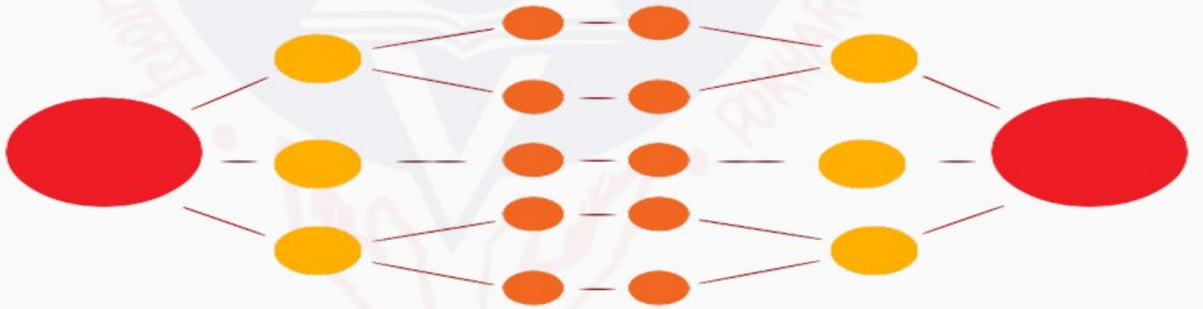
1. Divide and Conquer
2. Greedy Algorithms
3. Backtracking Algorithm
4. Branch and Bound
5. Randomized Algorithm
6. Dynamic Programming
7. Brute-force
8. Approximation Algorithm



CamScanner

Divide and Conquer

- Divide and conquer is a powerful strategy for designing efficient algorithms.
- It breaks the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively (or iteratively), and then combine these solutions to create a solution to the original problem
- It divides a problem until the subproblem is smaller and simpler enough (base case) that we can solve directly.



CamScanner

Divide and Conquer

- **Divide**
  - Decompose the problem into one or more subproblems that are smaller instances of the same problem
  - Keep breaking the subproblems until no subproblem is further divisible .i.e break the problem until we get the subproblem which can solve directly.
  - It generally follows recursive approach but iterative approach can also be used.
- **Conquer**
  - Solve successively and independently each of the subproblems (recursively or iteratively)
- **Combine**
  - Combine the subsolutions of subproblems to obtain the solution to the original problem



CamScanner

Divide and Conquer

```
1   Algorithm DAndC(P)
2   {
3       if Small(P) then return S(P);
4       else
5       {
6           divide P into smaller instances P_1, P_2, ..., P_k, k ≥ 1;
7           Apply DAndC to each of these subproblems;
8           return Combine(DAndC(P_1),DAndC(P_2),...,DAndC(P_k));
9       }
10  }
```

Algorithm 3.1 Control abstraction for divide-and-conquer

- P is the problem to be solved.
- Small(P) is a smaller subproblem which solution is S(P).



CamScanner

Divide and Conquer Problems

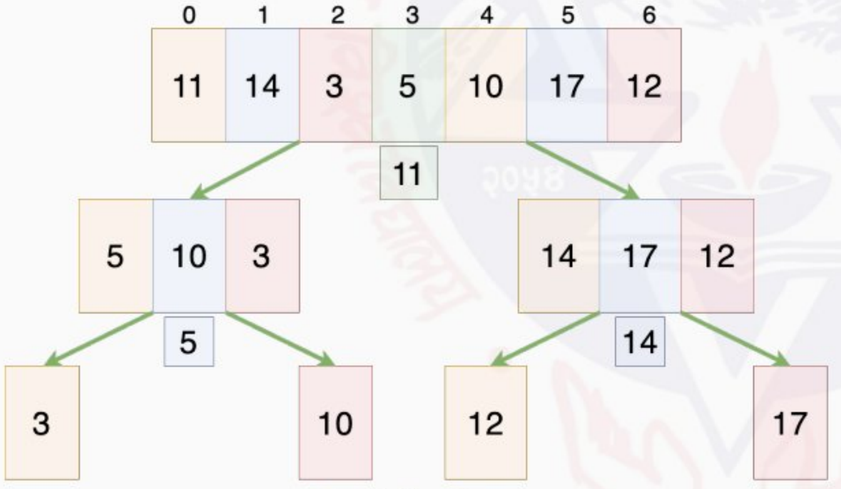
- Quick Sort
- Merge Sort
- Binary Search



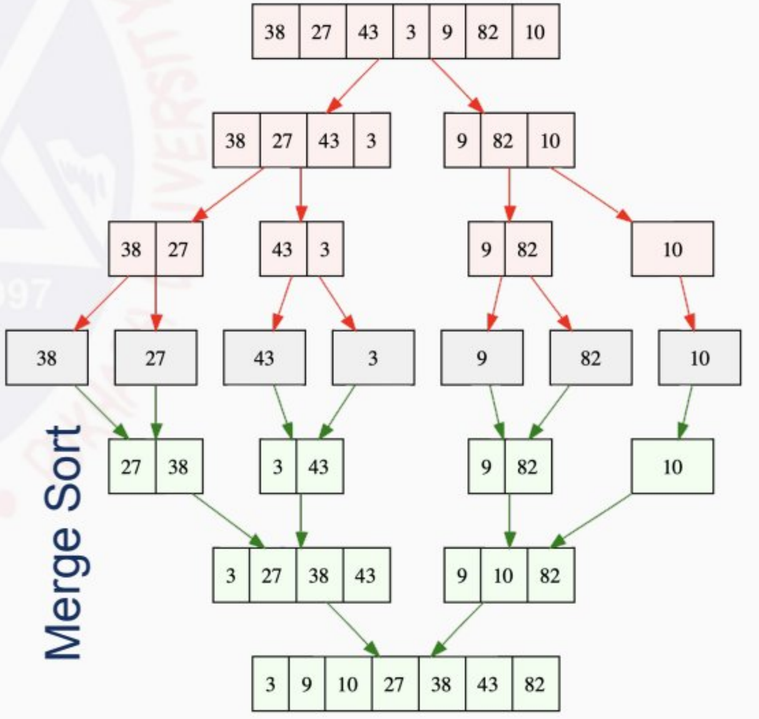
CamScanner

Divide and Conquer: Quick sort

- Quick sort is the fastest known general-purpose in-memory sorting algorithm in the average case.
- It does not require the extra array as needed by Merge sort, so it is space efficient as well.



Quick Sort



Merge Sort

Divide and Conquer: Quick sort

- Quick sort work as follows:
  - It first chooses as a pivot element from the items in the array to be sorted.
  - The array is then partitioned on either side of the pivot: elements are moved so that those greater than the pivot are to its right, whereas the others are to its left.
  - If now the sections of the array on either side of the pivot are sorted independently by recursive calls of the algorithm, the final result is a completely sorted array, no subsequent merge step being necessary.
- To balance the sizes of the two sub-instances to be sorted, we would like to use the median element as the pivot.
- Unfortunately, finding the median takes more time than it is worth. For this reason we simply use an arbitrary element of the array as the pivot, hoping for the best.



CamScanner

Divide and Conquer: Quicksort

- Quick sort applies the divide-and-conquer process for sorting an array A[p:r] :
- **Divide**
  - Partition (rearrange) the array into two subarrays A[p:q-1] (the ***low*** side) and A[q+1:r] (the ***high*** side) such that each element in the ***low*** side of the partition is less than or equal to the pivot and each element in the ***high*** side is greater than the pivot.
  - Compute the index of the pivot as part of this partitioning procedure.
- **Conquer**
  - Conquer by calling quicksort recursively to sort each of the subarrays- A[p:q-1] and A[q+1:r]
- **Combine**
  - Nothing to combine. Since the two subarrays are already sorted, no work is needed to combine them. All the elements in A[p:q-1] are sorted and less than or equal to A[q]. All the elements in A[q+1:r] are also sorted and greater than or equal to A[q].

CamScanner

Quicksort: Algorithm

Algorithm QUICKSORT(A, p, r)

1. **if** $p < r$ **then**
2. $\quad q \leftarrow$ PARTITION(A, p, r)
3. $\quad$ QUICKSORT$(A, p, q-1)$
4. $\quad$ QUICKSORT$(A, q+1, r)$

Algorithm PARTITION(A, p, r)

1. $pivot \leftarrow A[r]$
2. $i \leftarrow p-1$
3. **for** $j \leftarrow p$ **to** $r-1$ **do**
4. $\quad$ **if** $A[j] \leq pivot$ **then**
5. $\quad\quad i \leftarrow i+1$
6. $\quad\quad$ exchange $A[i]$, $A[j]$
7. exchange $A[i+1]$, $A[r]$
8. **return** $i+1$

CamScanner

Implementation of Quick Sort

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    // i= index of the smaller element initially set to -1
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    } // Swap the pivot element to its correct position
    swap(arr[i + 1], arr[high]);
    return i + 1; // Return the pivot index
}
```

```
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```



CamScanner

Implementation of Quick Sort

```
void quickSort(int arr[], int low, int high) {
    //if the array contains more than one element
    if (low < high) {
        // 1. Partition the array
        int pivotIndex = partition(arr, low, high);
        // 2. Call quicksort for the left part
        quickSort(arr, low, pivotIndex - 1);
        // 3. Call quicksort for the right part
        quickSort(arr, pivotIndex + 1, high);
    }
}
```



CamScanner

Analysis of Quick Sort

- The running time of quick sort is equal to the running time of the two recursive calls plus the linear time spent in the partition
- This gives the basic quick sort relation

$$T(N) = T(q) + T(N - q - 1) + cN$$

Here,

T(***N***) = Total cost of the quick sort

N = total number of elements in the array

q = number of elements in first half

c***N*** = cost for partition algorithm (linear time spent in the partition)

c = a known constant

Algorithm QUICKSORT(A, p, r)

1. **if** $p < r$ **then**
2. $q \leftarrow$ PARTITION(A, p, r)
3. QUICKSORT$(A, p, q-1)$
4. QUICKSORT$(A, q+1, r)$

CamScanner

Analysis of Quick Sort: Best Case

- We have

$$T(N) = 2T(N/2) + cN$$

Divide the both sides by N

$$T(N) / N = T(N/2) / N/2 + c$$

$$= T(N/4) / N/4 + c + c$$

$$= T(N/8) / N/8 + c + c + c$$

...

$$= T(1) / 1 + c \log N$$

Therefore, $T(N) = N + cN \log N$

If the pivot divides the array into two nearly equal halves, then:

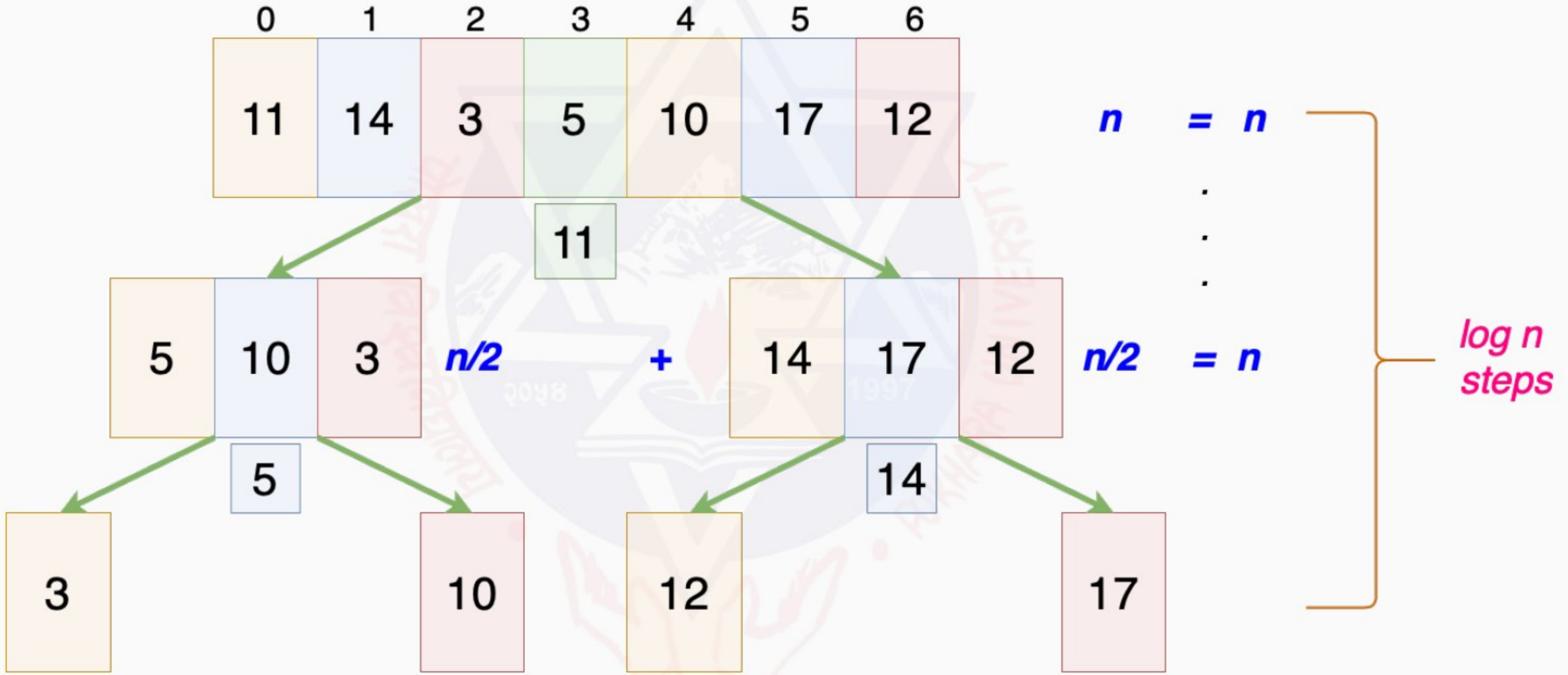
- At the first level, n comparisons are made (partitioning the entire array).
- At the second level, each half (size n / 2) requires n / 2 + n / 2 = n comparisons.
- This process continues for log n levels.

In each recursive call, the pivot divides subarray into nearly equally half. Therefore, there are only ***log N*** steps needed for sorting whole array.



CamScanner

Analysis of Quick Sort: Best Case



Analysis of Quick Sort: Worst Case

- The worst-case scenario occurs when the pivot is always the smallest or largest element.
- This leads to one side of the partition having $n - 1$ elements and the other side having 0 elements.
- We have,

$$T(N) = T(q) + T(N - q - 1) + cN$$

- For worst case $q = 0$, therefore we get

$$T(N) = T(0) + T(N - 1) + cN$$

$$\approx T(N - 1) + cN$$

Therefore, $T(N) \approx T(N - 1) + cN$

CamScanner

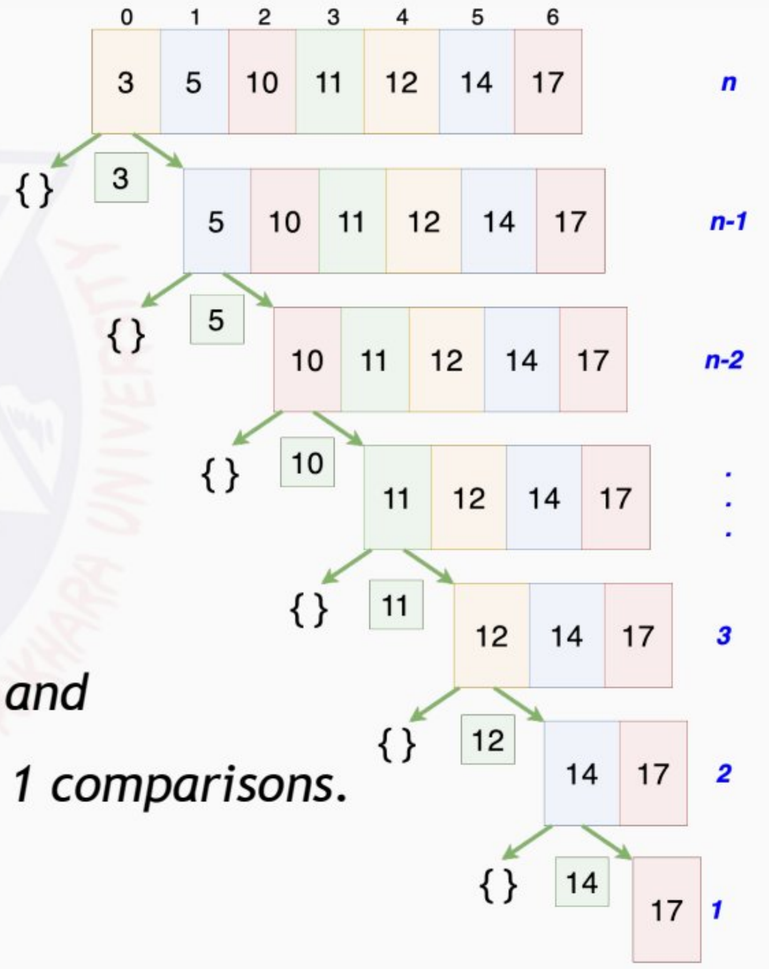
Analysis of Quick Sort: Worst Case

- We have

$$T(N) = T(N-1) + cN$$
$$= T(N-2) + c(N-1) + cN$$
$$= T(N-3) + c(N-2) + c(N-1) + cN$$
$$\ldots$$
$$T(N) = T(1) + c\sum_{i=2}^{N} i = \Theta(N^2)$$

In this case, the recursive calls occur n times, and

at each level, we perform n, n – 1, n – 2, ..., 1 comparisons.



Analysis of Quick Sort: Average Case

Class Discussion



CamScanner

Analysis of Quick Sort

Case	Time Complexity	Space Complexity
Best Case	$O(n \log n)$	$O(\log n)$
Average Case	$O(n \log n)$	$O(\log n)$
Worst Case	$O(n^2)$	$O(n)$



CamScanner

Choosing a Pivot Element

- Choosing a pivot element is a critical step in Quick Sort because it directly affects the efficiency of the algorithm.
- The goal is to select a pivot that divides the array into two nearly equal-sized partitions, minimizing the depth of recursion and achieving optimal time complexity (O (n log n)).
    - First Element as a Pivot element
    - Last element as a pivot element
    - Random pivot
    - Median-of-Three pivot
    - Median of the array (Optimal Pivot)



CamScanner

Choosing a Pivot Element

- **First Element as Pivot:**
  - Choose the first element of the array/subarray as the pivot.
  - Example:
    - Array: [8 , 2 , 6 , 9 , 3]
    - Pivot: 8
  - Advantages:
    - Simple to implement. No additional overhead.
  - Disadvantages:
    - Performs poorly on sorted or reverse-sorted arrays (worst case: $O(n^2)$).



CamScanner

Choosing a Pivot Element

- **Last Element as Pivot:**
  - Choose the last element of the array/subarray as the pivot.
  - Example:
    - Array: [8 , 2 , 6 , 9 , 3]
    - Pivot: 3
  - Advantages:
    - Simple to implement.
  - Disadvantages:
    - Performs poorly on sorted or reverse-sorted arrays (worst case: $O(n^2)$).



CamScanner

Choosing a Pivot Element

- **Random Pivot:**
  - Choose a random element of the array/subarray as the pivot.
  - Example:
    - Array: [8 , 2 , 6 , 9 , 3]
    - Pivot: 6 (randomly chosen)
  - Advantages:
    - Reduces the likelihood of worst-case scenarios, as the pivot is unpredictable.
    - Ensures good average-case performance.
  - Disadvantages:
    - Requires generating random numbers, which adds a slight computational overhead.



CamScanner

Choosing a Pivot Element

- **Median-of-three Pivot:**
  - Select the pivot as the median of the first, middle, and last elements of the array/subarray.
  - Example:
    - Array: [8 , 2 , 6 , 9 , 3]
    - First: 8, Middle: 6, last: 3
    - Pivot: Median of [8, 6, 3] = 6
  - Advantages:
    - Improves partitioning for sorted or nearly sorted arrays.
    - Reduces the chance of unbalanced partitions, improving performance.
  - Disadvantages:
    - Slightly more complex to implement due to the calculation of the median.



CamScanner

Choosing a Pivot Element

- **Median of the array (Optimal Pivot):**
  - Choose the median of the entire array as the pivot.
  - Example:
    - Array: [8 , 2 , 6 , 9 , 3]
    - Median: 6
  - Advantages:
    - Guarantees the best possible partitions, minimizing recursion depth.
    - Ensures $O(nlogn)$ time complexity in all cases.
  - Disadvantages:
    - Finding the median requires $O(n)$ time, which negates the benefits of Quick Sort's simplicity. Rarely used in practice for this reason.



CamScanner

Merge Sort

- The *merge sort* algorithm closely follows the divide-and-conquer method.
- In each step, it sorts a subarray , starting with the entire array and recursing down to smaller and smaller subarrays. Here is how merge sort operates:

Divide the subarray $A[p:r]$ to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint q of $A[p:r]$ (taking the average of p and r), and divide $A[p:r]$ into subarrays $A[p:q]$ and $A[q+1:r]$.

Conquer by sorting each of the two subarrays $A[p:q]$ and $A[q+1:r]$ recursively using merge sort.

Combine by merging the two sorted subarrays $A[p:q]$ and $A[q+1:r]$ back into $A[p:r]$, producing the sorted answer.



CamScanner

Merge Sort

- The *merge sort* reaches the base case when $p = r$ (i.e. the subarray $A[p : r]$ to be sorted has just one element.
- The key operation of the merge sort algorithm occurs in the **combine** step, which merges two adjacent, sorted subarrays using $Merge(A, p, q, r)$.

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```



CamScanner

Merge Sort

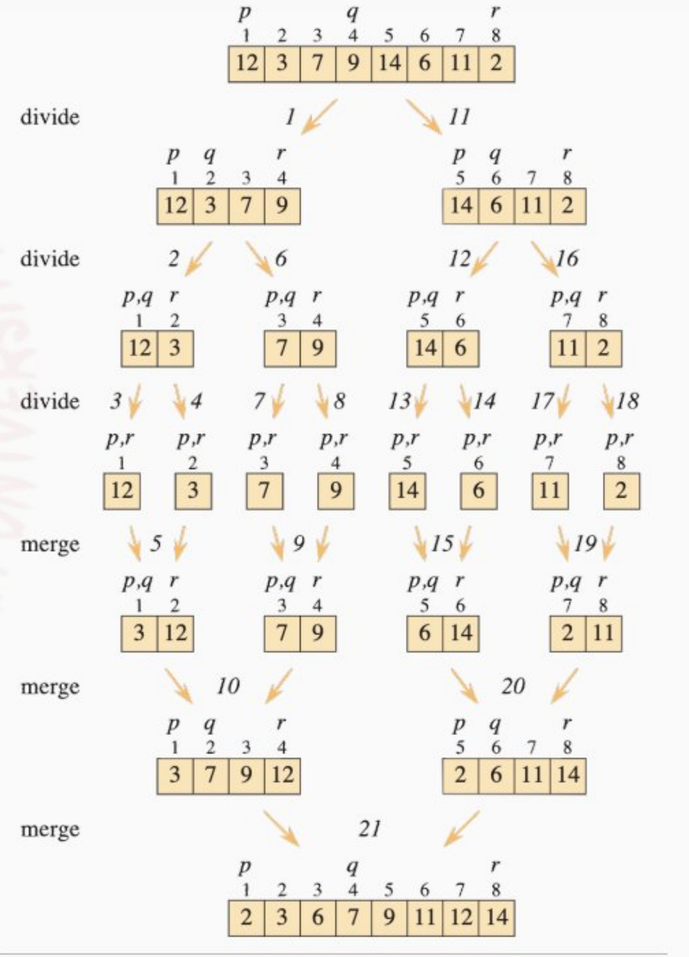
- The $Merge(A, p, q, r)$ works as:
  - Let's think about how long it takes to merge two sorted piles of cards.
  - Each basic step takes constant time, since you are comparing just the two top cards.
  - If the two sorted piles that you start with each have $n/2$ cards, then the number of basic steps is at least $n/2$ and at most n.
  - With each basic step taking constant time and the total number of basic steps being between $n/2$ and n, we can say that merging takes time roughly proportional to n. That is, merging takes time $\Theta(n)$.
- This function can be written in recursively but always written using iteration to get better performance.

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //      copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //      remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```


CamScanner

Analysis of Merge Sort

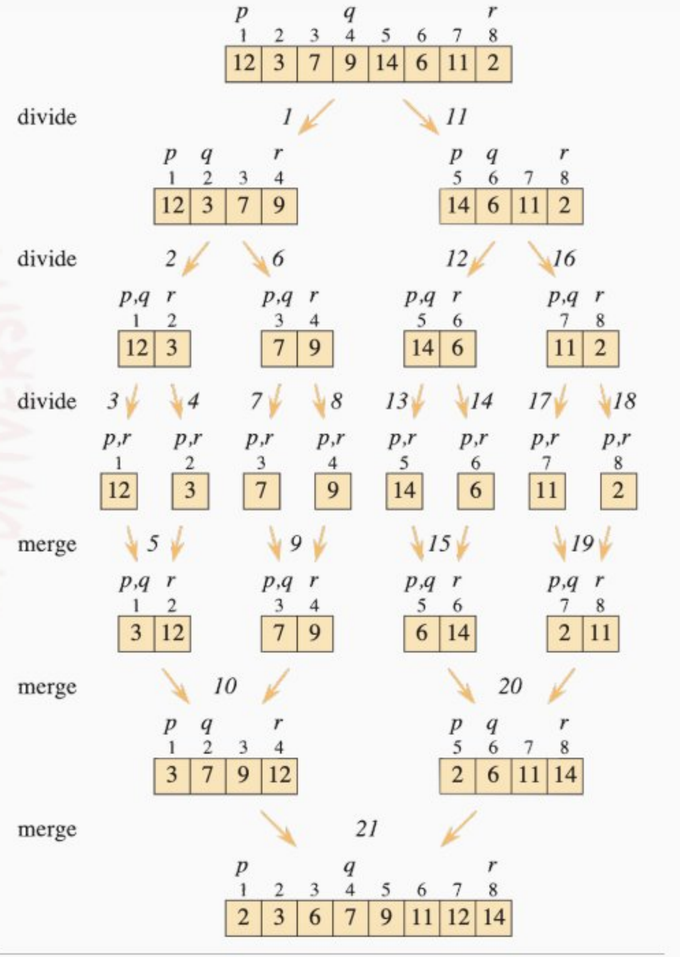
- let be the worst-case running time on a problem of size . If the problem size is small enough, say for some constant , the straightforward solution takes constant time, which we write as .[16] Suppose that the division of the problem yields subproblems, each with size , that is, the size of the original. For merge sort, both and are , but we'll see other divide-and-conquer algorithms in which . It takes time to solve one subproblem of size , and so it takes time to solve all of them. If it takes time to divide the problem into subproblems and time to combine the solutions to the subproblems into the solution to the original problem, we get the recurrence



Analysis of Merge Sort

- Let $T(n)$ be the worst case running time on a problem of size n.
- If the problem size is small enough, say $n < n_0$ for some constant $n_0 > 0$, the solution takes a constant time $\Theta(1)$
- Suppose the problem is divided into a subproblems, each with size n/b.
- For merge sort $a = 2$ and $b = 2$
- Merge sort takes $T(n/b)$ time to solve one subproblem, then if it takes $D(n)$ time to dived the problem and $C(n)$ time to combine the solutions of subproblems, $T(n)$ is given by

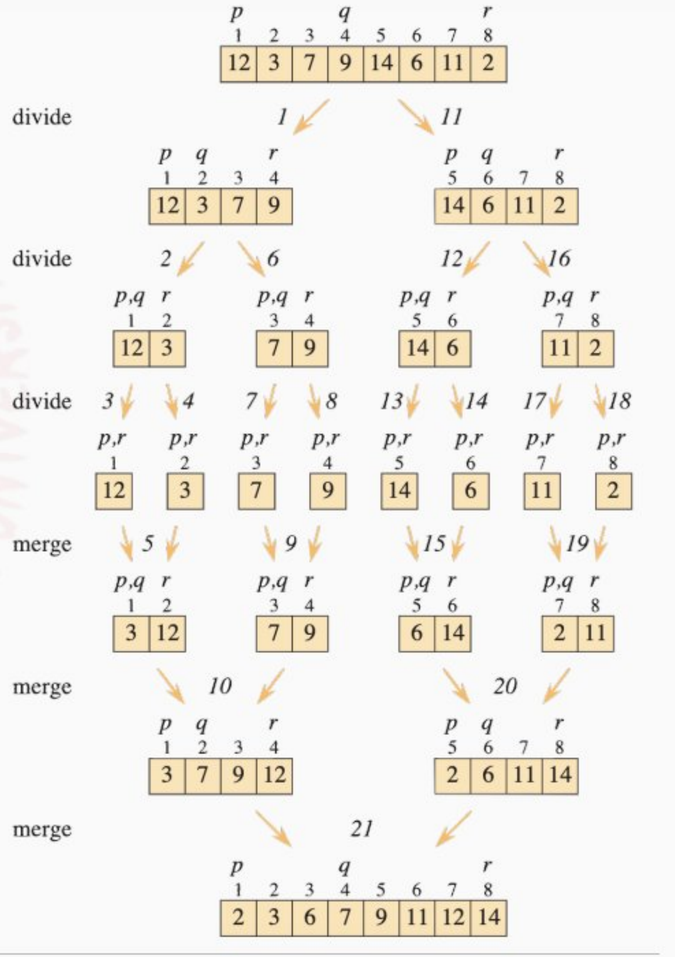
$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0 , \\ D(n) + aT(n/b) + C(n) & \text{otherwise} . \end{cases}$$



Analysis of Merge Sort (Worst Case)

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0 \ , \\ D(n) + aT(n/b) + C(n) & \text{otherwise} \ . \end{cases}$$

- **Divide**: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.
- **Conquer**: Recursively solving two subproblems, each of size $n/2$, contributes $2T(n/2)$ to the running time
- **Combine**: Since the MERGE procedure on an n-element subarray takes $\Theta(n)$ times, we have $C(n) = \Theta(n)$.
- Hence, $D(n) + C(n) = \Theta(n)$
- Therefore, in worst case, $T(n) = 2T(n/b) + \Theta(n)$
- $T(n) = n \log n$



Analysis of Merge Sort (Worst Case)

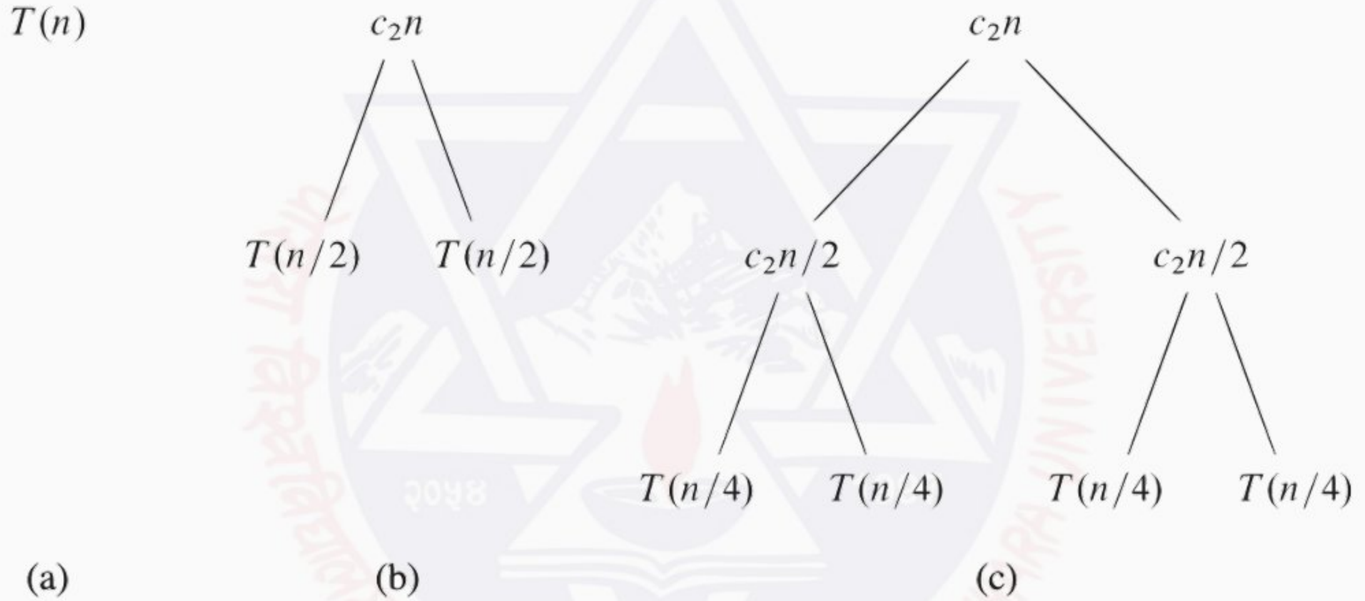


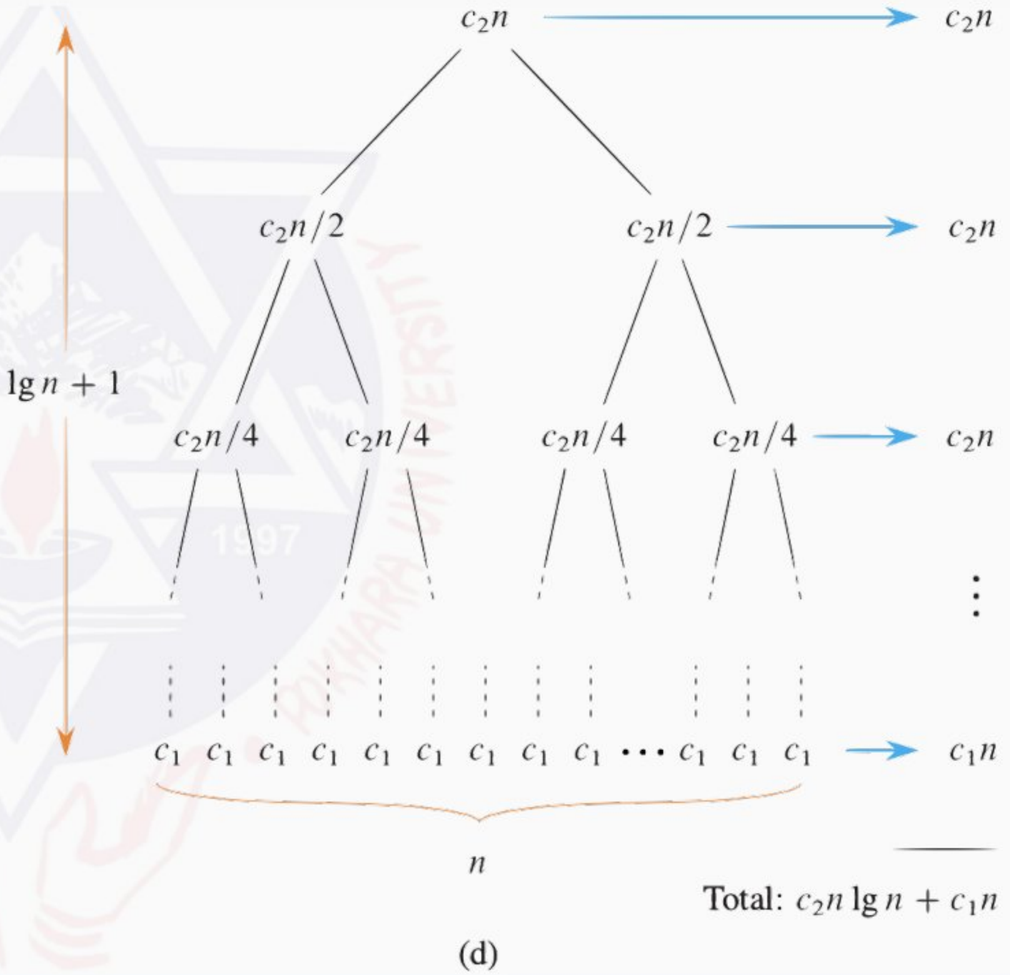
Figure 2.5 How to construct a recursion tree for the recurrence (2.4). Part **(a)** shows $T(n)$, which progressively expands in **(b)–(d)** to form the recursion tree. The fully expanded tree in part (d) has $\lg n + 1$ levels. Each level above the leaves contributes a total cost of $c_2 n$, and the leaf level contributes $c_1 n$. The total cost, therefore, is $c_2 n \lg n + c_1 n = \Theta(n \lg n)$.

CamScanner

Analysis of Merge Sort (Worst Case)

- For simplicity, assume that n is exact power of 2 and base case is $n = 1$, then

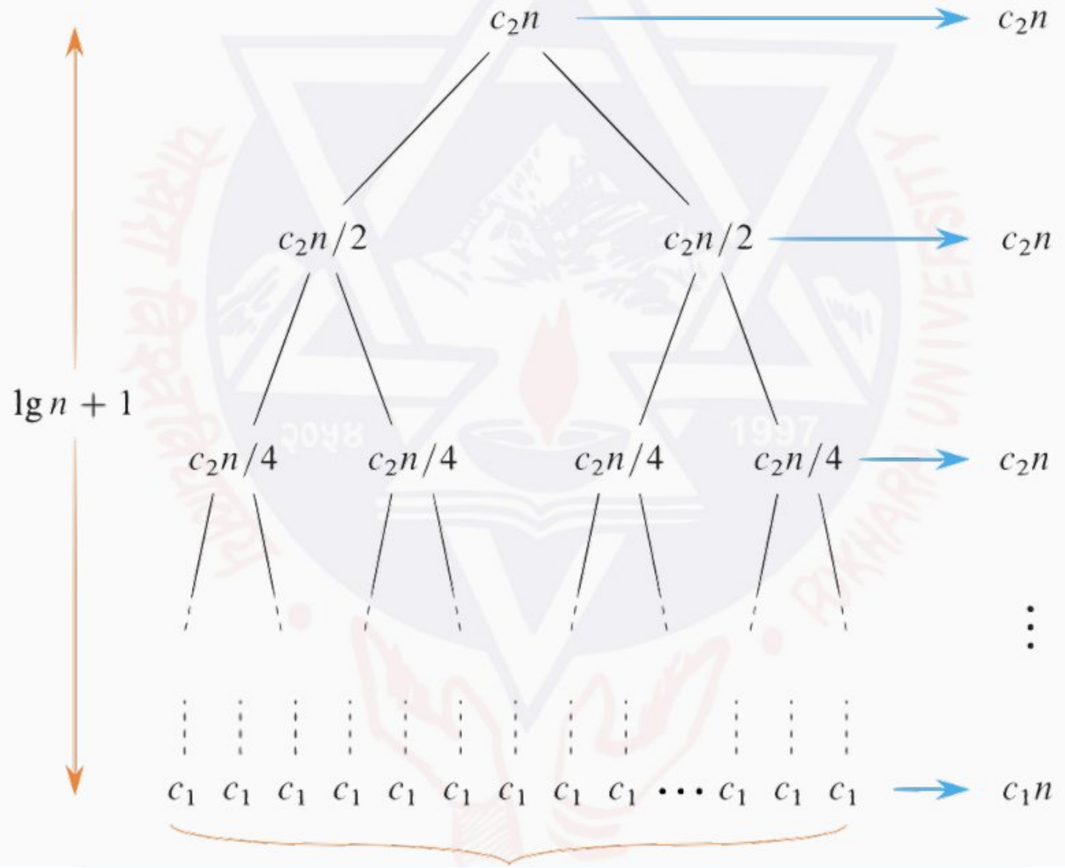
$$T(n) = \begin{cases} c_1 & \text{if } n = 1 , \\ 2T(n/2) + c_2 n & \text{if } n > 1 , \end{cases}$$



(d)

Analysis of Merge Sort (Best Case)

- In best case, $T(n) = n \log n$ (Class Discussion)



Average Case:
$T(n) = n \log n$

Binary Search

- Iterative Method

```
int binarySearch(int array[], int item, int low, int high) {
  // Repeat until the pointers low and high meet each other
  while (low <= high) {
    int mid = low + (high - low) / 2;
    if (array[mid] == item)
      return mid;
    if (array[mid] < item)
      low = mid + 1;
    else
      high = mid - 1;
  }
  return -1;
}
```



CamScanner

Binary Search

- Recursive Method

```
int binarySearch(int array[], int item, int low, int high) {
  if (high >= low) {
    int mid = low + (high - low) / 2;
    // If found at mid, then return it
    if (array[mid] == item)
      return mid;
    // Search the left half
    if (array[mid] > item)
      return binarySearch(array, item, low, mid - 1);
    // Search the right half
    return binarySearch(array, item, mid + 1, high);
  }
  return -1;
}
```



CamScanner

Analysis of Binary Search

- Best Case: $O(logn)$
- Worst Case:$O(n)$

Class Discussion



CamScanner

Greedy Algorithm

- Greedy algorithms work in phases.
- In each phase, a decision is made that appears to be good, without regard for future consequences. Generally, this means that some local optimum is chosen.
- This “take what you can get now” strategy is the source of the name for this class of algorithms.
- When the algorithm terminates, we hope that the local optimum is equal to the global optimum. If this is the case, then the algorithm is correct; otherwise, the algorithm has produced a suboptimal solution.
- If the absolute best answer is not required, then simple greedy algorithms are sometimes used to generate approximate answers, rather than using the more complicated algorithms generally required to generate an exact answer.



CamScanner

Greedy Algorithm

- Most of the problems have n inputs and require us to obtain a subset that satisfies some constraints.
- Any subset that satisfies these constraints is called a feasible solution.
- We need to find a feasible solution that either maximizes or minimizes the objective function. A feasible solution that does this is called an optimal solution.
- Generally provides usually an obvious way to find a feasible solution but not necessarily an optimal solution.



CamScanner

Greedy Algorithm

- Greedy algorithm is a straightforward algorithm design technique that works in stages, considering one input at a time.
- At each stage, a decision is made regarding whether or not a particular input is in an optimal solution. This is done by considering the inputs in an order determined by some selection procedure.
- If the inclusion of the next input, into the partially constructed optimal solution will result in an infeasible solution then this input is not added to the partial solution. Otherwise it is added.
- The selection procedure itself is based on some optimization measure. This measure is done by objective function.



CamScanner

Greedy Algorithm

- Greedy algorithm (**Subset paradigm**)
  - Several optimization measures are plausible (probable) for a given problem.
  - Most of them, however, will result in algorithms that generate sub-optimal solutions. This version of greedy technique is called subset paradigm.
  - Some problems like Knapsack, Job sequencing with deadlines and minimum cost spanning trees are based on subset paradigm.
- Greedy algorithm (**Ordering paradigm**)
  - For the problems that make decisions by considering the inputs in some order, each decision is made using an optimization criterion that can be computed using decisions already made. This version of greedy method is ordering paradigm.
  - Some problems like optimal storage on tapes, optimal merge patterns and single source shortest path are based on ordering paradigm.



CamScanner

Greedy Algorithm

```
1   Algorithm Greedy(a, n)
2   // a[1 : n] contains the n inputs.
3   {
4       solution := ∅; // Initialize the solution.
5       for i := 1 to n do
6       {
7           x := Select(a);
8           if Feasible(solution, x) then
9               solution := Union(solution, x);
10      }
11      return solution;
12  }
```

Algorithm 4.1 Greedy method control abstraction for the subset paradigm

CamScanner

Greedy Algorithm

CONTROL_ABSTRACTION_ORDERING(Elements, Criteria):

1. Initialize a solution set `S` (empty initially).
2. Sort the `Elements` based on the given `Criteria`.
3. FOR each element `E` in the sorted list:
    a. IF including `E` does not violate the constraints:
        i. Add `E` to the solution set `S`.
4. Return the solution set `S`.



CamScanner

How does Greedy Algorithm works?

- **Initialization**: Define the solution set (empty initially).
- Repeat until the entire problem is solved or no more choices remain
  - **Iterative Selection**: At each step, pick the best choice available based on a given criterion.
  - **Feasibility Check**: Ensure that the selected choice does not violate the problem's constraints.
  - **Solution Augmentation**: Add the chosen item to the solution set.



CamScanner

Greedy Algorithm

- Examples:
  - Activity Selection Problem
  - Knapsack Problem
  - Kruskal's Algorithm
  - Prim's Algorithm
  - Huffman Encoding



CamScanner

Activity Selection Problem

- Problem: Given ***n*** activities with start and finish times, select the maximum number of activities that don't overlap.

- Algorithm:
  1. Sort activities by their finish times.
  2. **Solution** = {}
  3. Iterate through the activities:
     1. Select an activity **A** if its start time is greater than or equal to the finish time of the previously selected activity.
     2. Add the activity to the Solution set: **Solution** ← **A**
  4. Return the **Solution**.



CamScanner

Activity Selection Problem

```
Algorithm ASP():
        Sort activities by finish time
        selected_activities = []
        current_time = 0

        for each activity:
            if activity.start_time >= current_time:
                selected_activities.append(activity)
                current_time = activity.finish_time

return selected_activities
```



CamScanner

Complexity of Activity Selection Problem

```
Algorithm ASP():
        Sort activities by finish time
        selected_activities = []
        current_time = 0

        for each activity:
            if activity.start_time >= current_time:
                selected_activities.append(activity)
                current_time = activity.finish_time

return selected_activities
```

O(n log n)



CamScanner

Complexity of Activity Selection Problem

```
Algorithm ASP():
    Sort activities by finish time
    selected_activities = []
    current_time = 0

    for each activity:
        if activity.start_time >= current_time:
            selected_activities.append(activity)
            current_time = activity.finish_time

return selected_activities
```

O(n)



CamScanner

Complexity of Activity Selection Problem

```
Algorithm ASP():
    Sort activities by finish time
    selected_activities = []
    current_time = 0

    for each activity:
        if activity.start_time >= current_time:
            selected_activities.append(activity)
            current_time = activity.finish_time

return selected_activities
```

Overall Complexity =
O(n log n)



CamScanner

Activity Selection Problem: Implementation

```
void activitySelection(vector<Activity>& activities) {
    // Step 1: Sort activities by finish time
    sort(activities.begin(), activities.end(), compare);

    cout << "Selected activities: " << endl;

    // Step 2: Select the first activity
    int lastFinishTime = activities[0].finish;
    cout << "(" << activities[0].start << ", "
         << activities[0].finish << ")" << endl;

    // Step 3: Iterate through the rest of the activities
    for (int i = 1; i < activities.size(); i++) {
        if (activities[i].start >= lastFinishTime) {
            // Select the activity
            cout << "(" << activities[i].start << ", "
                 << activities[i].finish << ")" << endl;
            lastFinishTime = activities[i].finish;
        }
    }
}
```



CamScanner

Knapsack Problem

- **Problem**:
  - You are given n objects and a knapsack. For $i = 1,2,\ldots,n,$ each object i has a positive weight w_i and a positive value v_i.
- **Constraint**:
  - The knapsack carry a weight not exceeding W.
- **Objective**:
  - To fill the knapsack in a way that maximizes the value of the included objects satisfying the capacity constraint.
- **Fractional Knapsack**:
  - The objects can be broken into smaller pieces, so we may decide to carry only a fraction x_i of object i, where $0 \leq x_i \leq 1$. In this case, the object i contributes $x_i w_i$ to the total weight in the knapsack and $x_i v_i$ to the value of the load.

Mathematically, maximize $\sum_{n=i}^{n} x_i v_i$ such that $\sum_{n=i}^{n} x_i w_i \leq W.$

CamScanner

Knapsack Problem

- **Algorithm for fractional knapsack problem:**

    1. **Calculate** value-to-weight ratio $\frac{v_i}{w_i}$
    2. **Sort** n objects in descending order based on the value-to-weight ratio
    3. **Initialize**, value=0
    4. For each **object** i**:**
        1. If $w_i \leq W$, include the full object
            1. $W = W - w_i$
            2. value = value + v_i
        2. Otherwise, include a fraction of the object
            1. value = value + $v_i * \frac{W}{w_i}$
            2. Break (Since knapsack is full in this stage)



CamScanner

Fractional Knapsack: Implementation

```cpp
// Implementing Fractional Knapsack using Greedy Algorithm
double fractionalKnapsack(vector<Item> items, int W) {
    // 1. Sort items by value-to-weight ratio
    sort(items.begin(), items.end(), compare);

    // 2. Initialize the value in the knapsack
    double totalValue = 0.0;

    // 3. Iterate through the sorted items
    for (Item item : items) {
        if (W >= item.weight) {
            // Take the full item
            totalValue += item.value;
            W -= item.weight;
        } else {
            // Take a fraction of the item
            totalValue += item.value * ((double)W / item.weight);
            break; // Knapsack is full
        }
    }

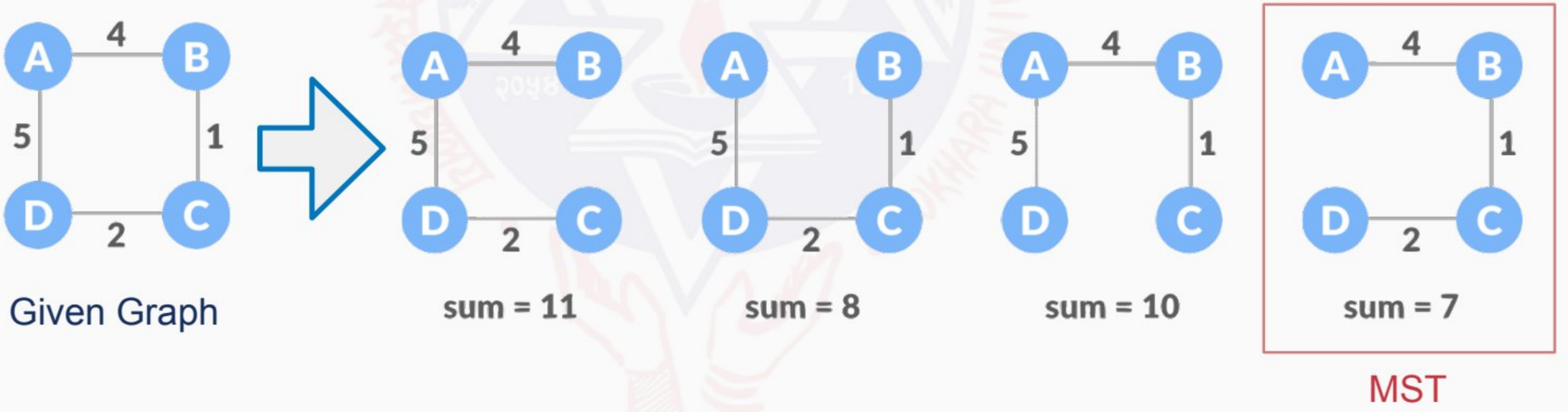
    return totalValue;
}
```



CamScanner

Kruskal's Algorithm

- **Minimum Spanning Tree (MST):** The minimum spanning tree T for a graph G=(V,E) is defined as.
  - T is an acyclic subset of E that connects all the vertices in V.
  - The sum of the weights of the edges in T is minimized.



Kruskal's Algorithm

- **Kruskal's Algorithm**
  - is also used to find the minimum spanning tree for a connected weighted graph.
- It also follows the **greedy approach**
  - It finds an optimum solution at every stage instead of focusing on a global optimum.
- Kruskal's algorithm
  - It starts from edges with the lowest weight and
  - keeps adding the edges until the final minimum spanning tree is obtained.



CamScanner

Kruskal's Algorithm

- **Kruskal's Algorithm:**

1. Sort all the edges from low weight to high.
2. Pick the edge with the lowest weight.
3. If the edge to be added doesn't create a cycle, add it to the spanning tree. Otherwise, reject the edge.
4. Repeat step 2 to 3 to add the edges until we reach all vertices, and a minimum spanning tree is created.



CamScanner

Kruskal's Algorithm

```
Kruskal(Graph):
    MST = []                          # List to store edges in the MST
    TotalWeight = 0                   # Variable to store total weight of MST
    DisjointSet = MakeSet(V)          # Initialize union-find data structure for vertices

    Edges = Graph.Edges               # Get all edges in the graph
    Sort(Edges) by weight             # Sort edges in non-decreasing order of weight

    for Edge in Edges:
        u, v, weight = Edge           # Extract vertices and weight of the edge
        if Find(DisjointSet, u) != Find(DisjointSet, v):
            Union(DisjointSet, u, v)  # Merge the two sets
            MST.append(Edge)          # Add edge to MST
            TotalWeight += weight     # Add edge weight to total weight

        if len(MST) == V - 1:         # Stop if MST has V - 1 edges
            break

    return MST, TotalWeight
```

Worst-case: $O(ElogE)$

Average-case: $O(VlogE)$

CamScanner

Backtracking Algorithm

- Many problems which deal with searching a set of solutions or which ask for an optimal solution satisfying some constraints **can be solved using backtracking**.
- **Desired Solution:**
  - is an n-tuple $(x_1, x_2, \ldots, x_n)$, where the x_i are chosen from some finite set S_i.
  - finding one ***vector*** that maximizes or minimizes or satisfies a ***criterion*** function $P(x_1, x_2, \ldots, x_n)$.
- **Basic Idea**:
  - Its basic idea is to build up the solution vector one component at a time using ***criterion*** function $P_i(x_1, x_2, \ldots, x_i)$ to test whether the solution vector being formed has any chance of success.
  - As soon as it determines that the partial solution vector $(x_1, x_2, \ldots, x_i)$ can in no way lead to an optimal solution, it then entirely ignores the $m_{i+1}, \ldots, m_n$ possible test vectors.



CamScanner

Backtracking Algorithm

- A practical example of a backtracking algorithm is the problem of arranging furniture in a new house. There are many possibilities to try, but typically only a few are actually considered. Starting with no arrangement, each piece of furniture is placed in some part of the room. If all the furniture is placed and the owner is happy, then the algorithm terminates. If we reach a point where all subsequent placement of furniture is undesirable, we have to undo the last step and try an alternative. Of course, this might force another undo, and so forth. If we find that we undo all possible first steps, then there is no placement of furniture that is satisfactory. Otherwise, we eventually terminate with a satisfactory arrangement. Notice that although this algorithm is essentially brute force, it does not try all possibilities directly. For instance, arrangements that consider placing the sofa in the kitchen are never tried. Many other bad arrangements are discarded early, because an undesirable subset of the arrangement is detected. The elimination of a large group of possibilities in one step is known as **pruning.**

CamScanner

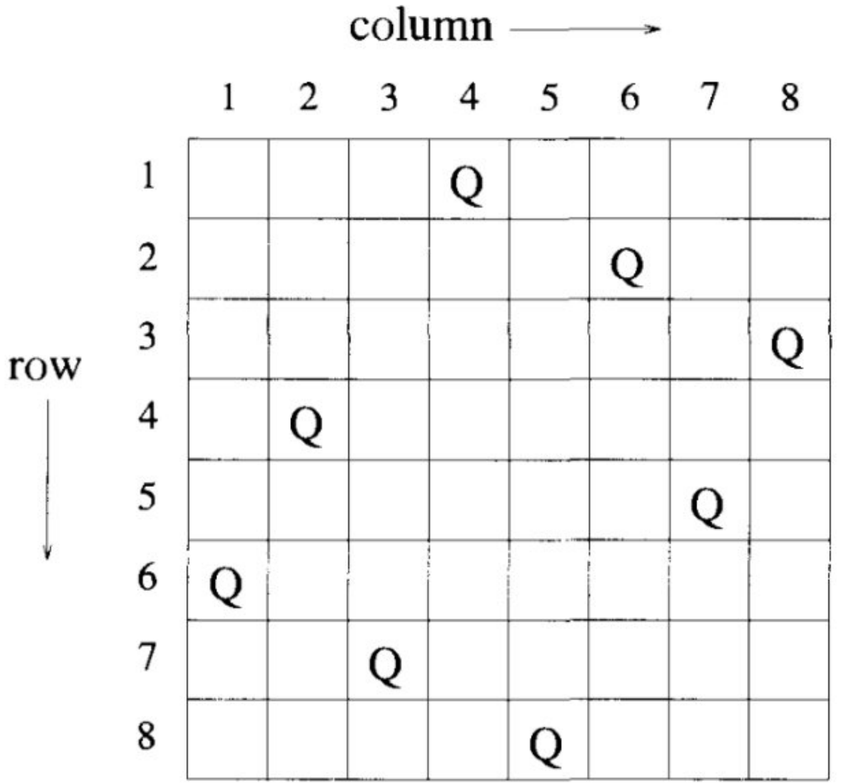
Backtracking Algorithm- 8 Queens Problem

- **8 Queens Problem**: A classical combinatorial problem is to place 8 queens on a $8X8$ chessboard so that no two queens attack each other (i.e. no two of them are on the same row, column or diagonal.
- Solution:
  - Let us number the rows and columns of chessboard through 1 to 8. Also number the queens through 1 to 8.
  - Since each queen must be on a different row, queen i must be placed on row i. So **all solutions can be represented as** 8-tuple $(x_1, x_2, \ldots, x_8)$ where x_i is the column on which queen i is placed.
  - **Constraints**:
    - The x_i are chosen from finite set $S_i = \{1,2,3,4,5,6,7,8\}$, $1 \leq i \leq 8$. Therefore the solution consists of 8^8 8-tuples
    - No two x_i can be the same (i.e all the queens must be on different columns) and no two queens can be on the same diagonal. This implies that all solutions are the permutation of the 8-tuples (i.e. the solution space is reduced from 8^8 tuples to $8!$ tuples.



CamScanner

Backtracking Algorithm- 8 Queens Problem



A 8-tuple Solution

(4,6,8,2,7,1,3,5)

Figure 7.1 One solution to the 8-queens problem

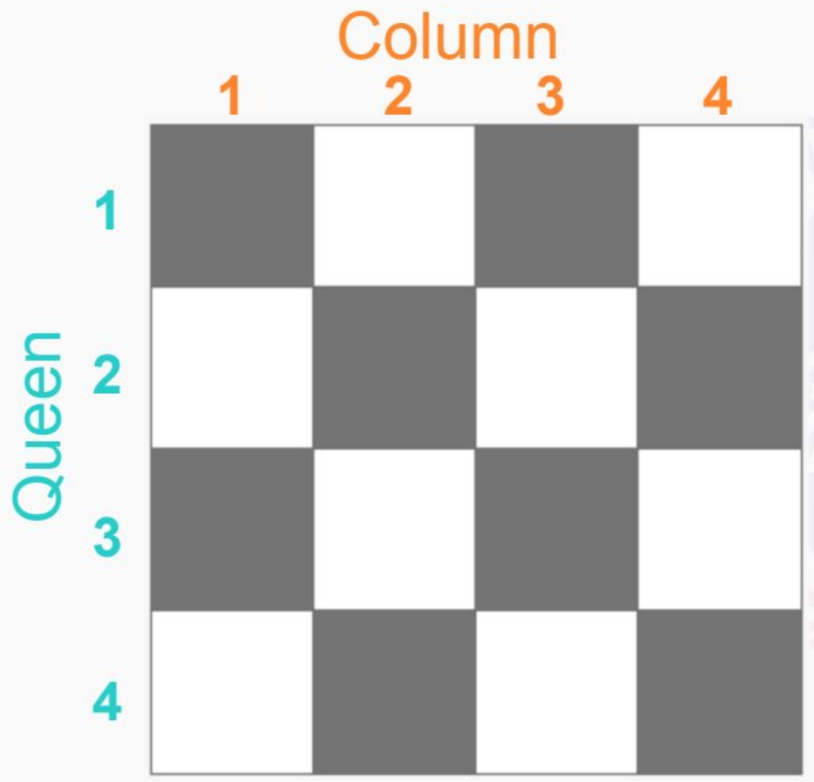
Backtracking Algorithm

- ***Live node:*** is a node which has been generated and all of which children have not yet been generated.
- ***E-node***: is the live node whose children are currently being generated. (It is a node being expanded)
- ***Dead node***: is a generated node which is not to be expanded further or all of whose children has been generated
- ***Bounding functions*** are used to kill live nodes without generating all their children
- Depth first node generation with bounding functions is called ***backtracking***.



CamScanner

Backtracking Algorithm- 4 Queens Problem



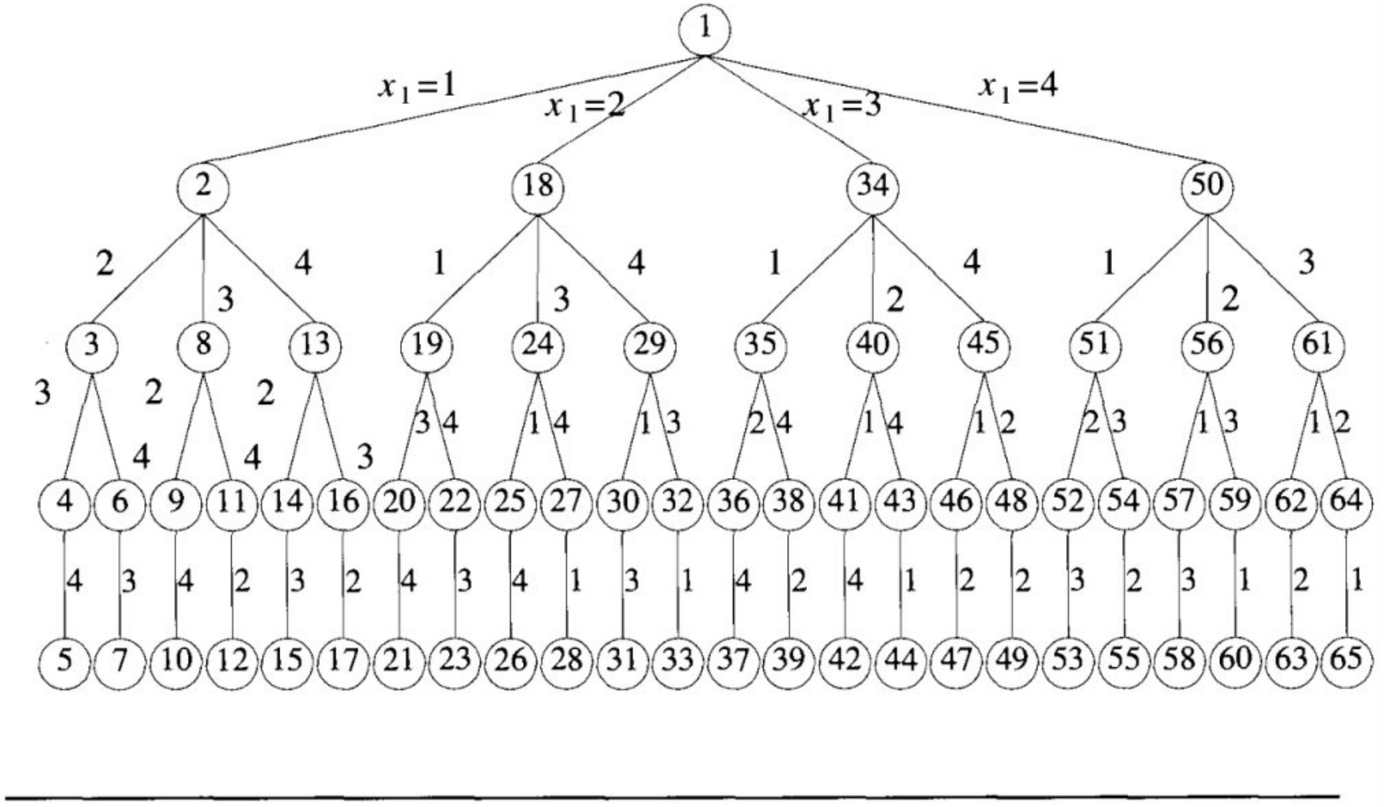
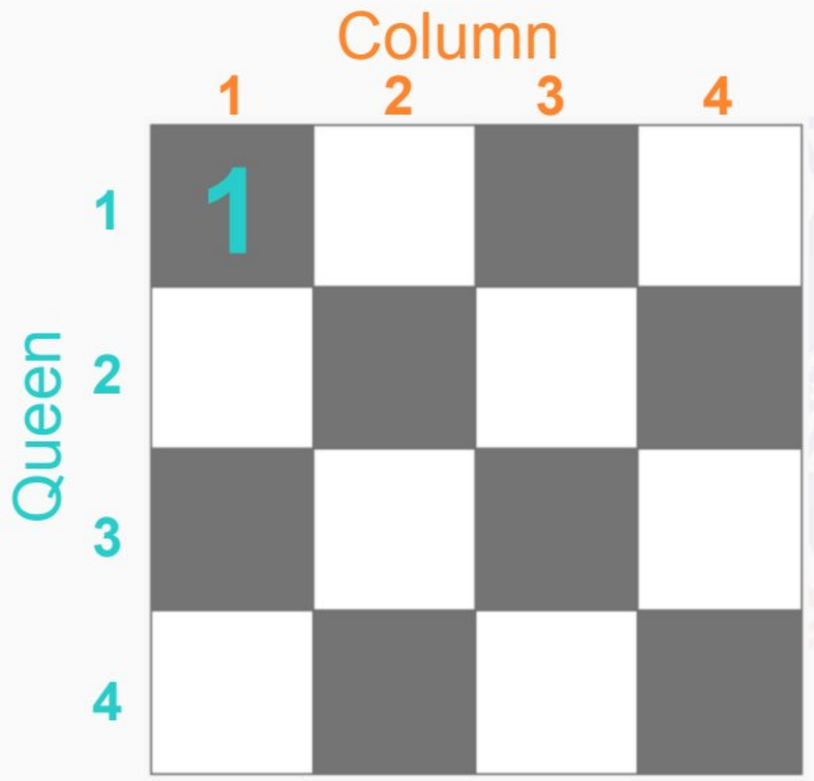


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



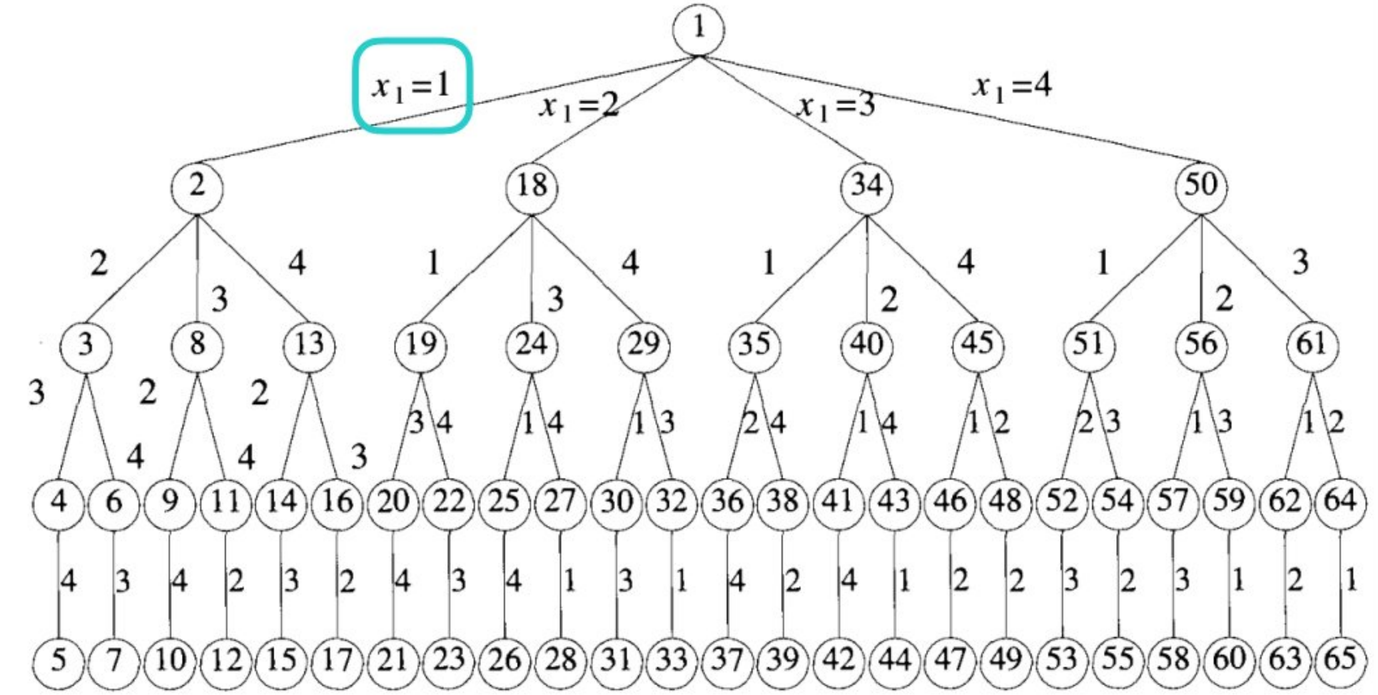
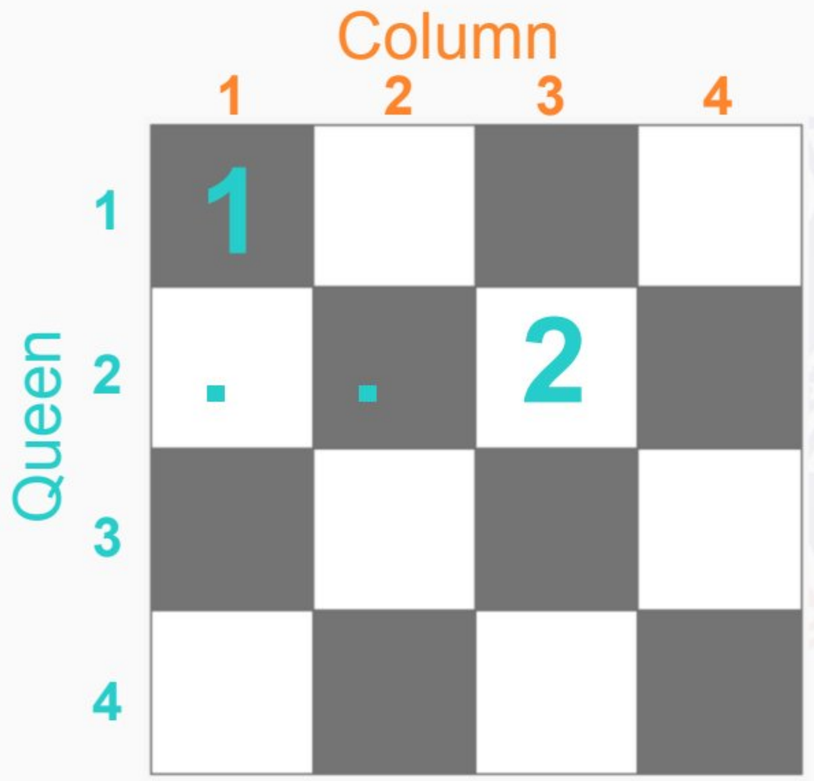


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



The dots indicate placement of queen which were tried and rejected since the another queen was attacking.

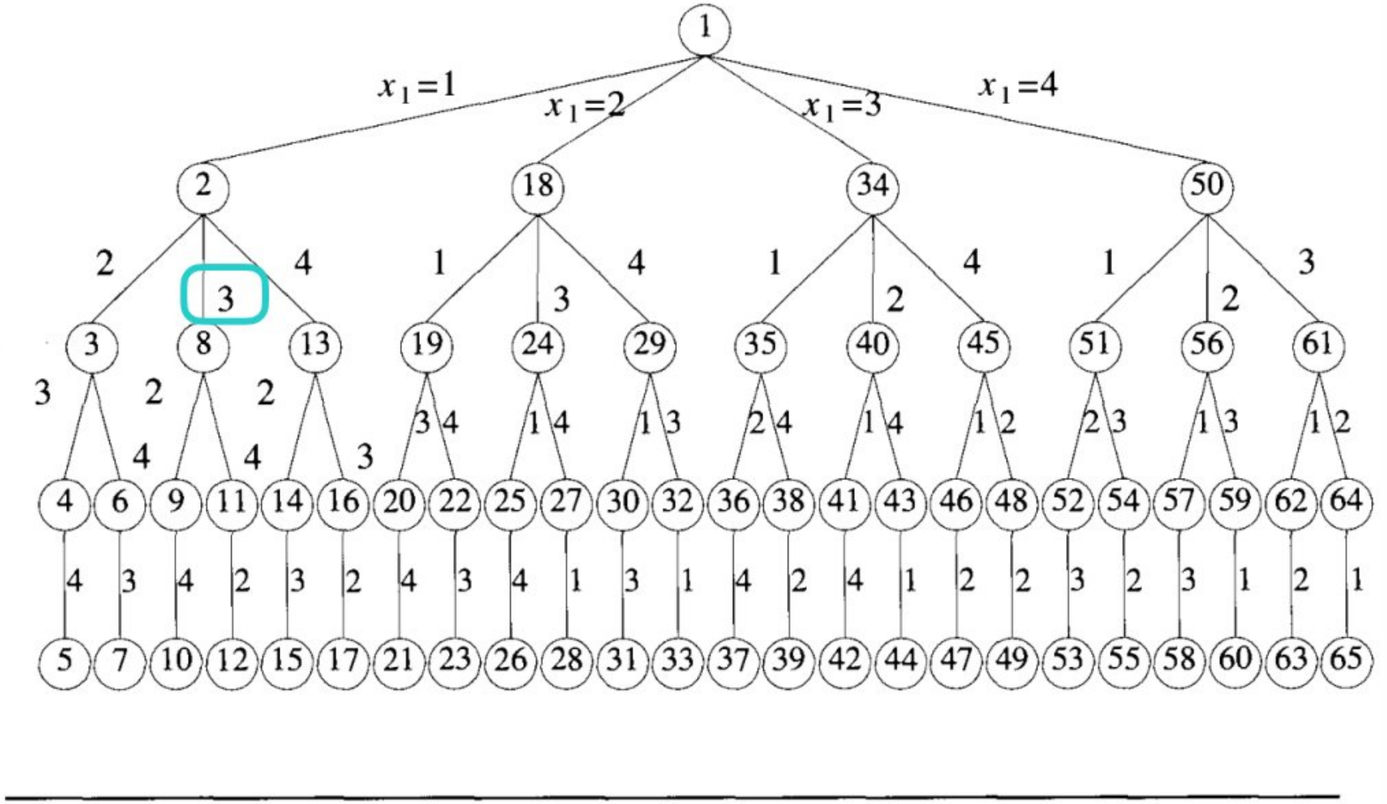
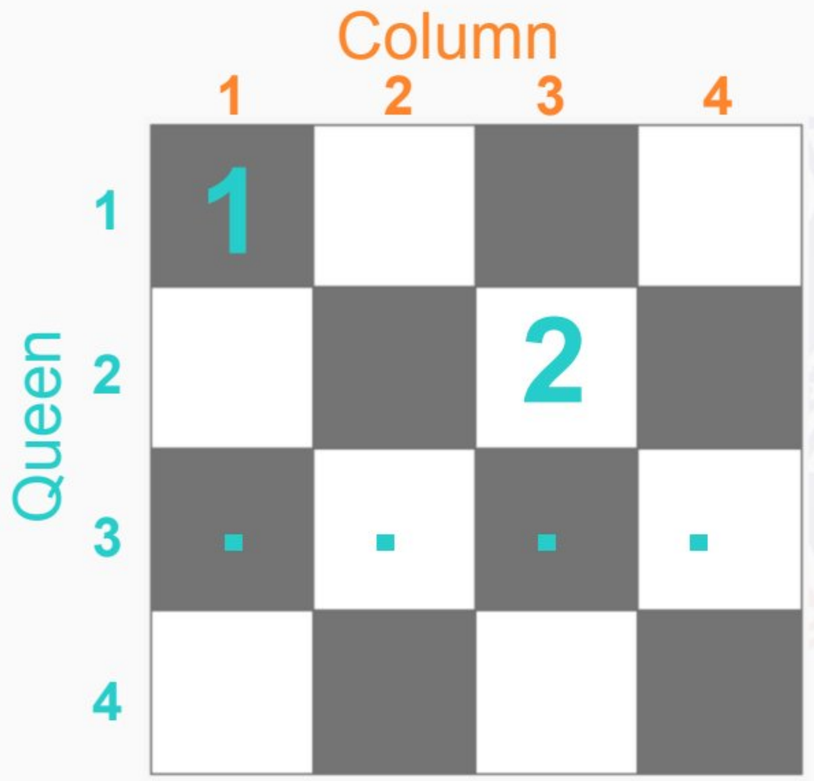


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



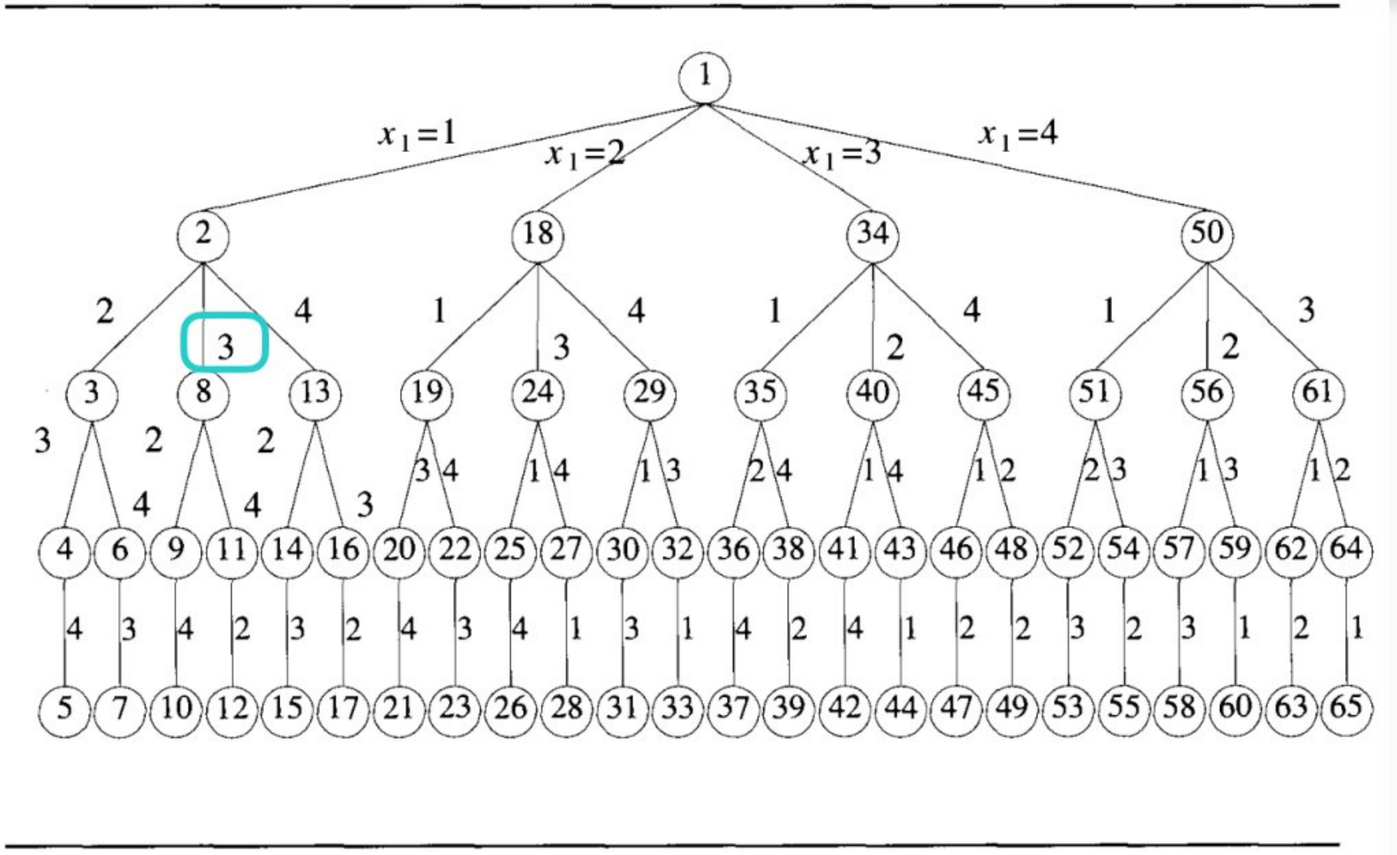
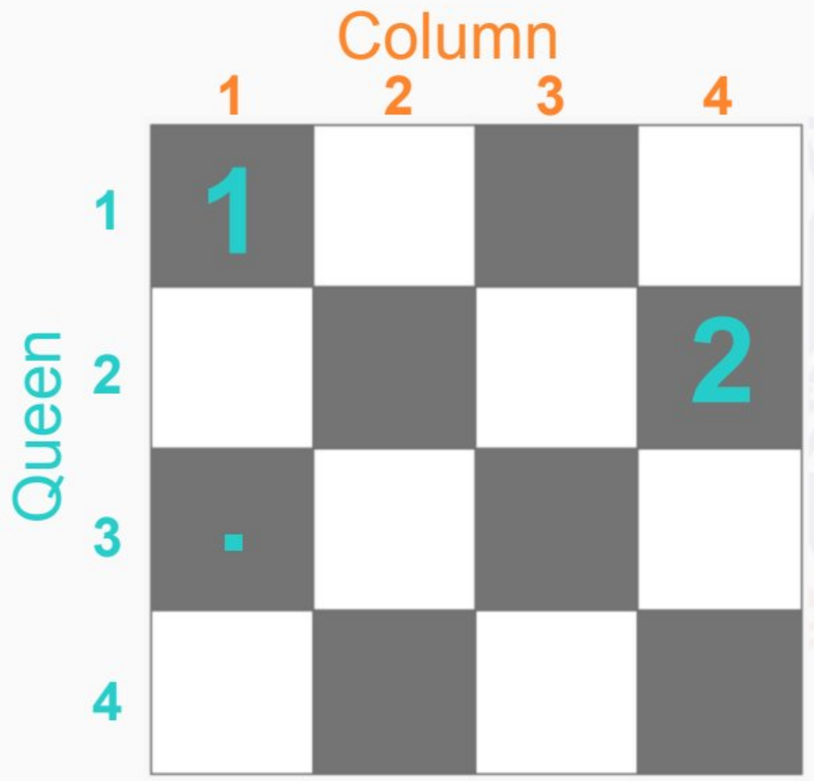


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



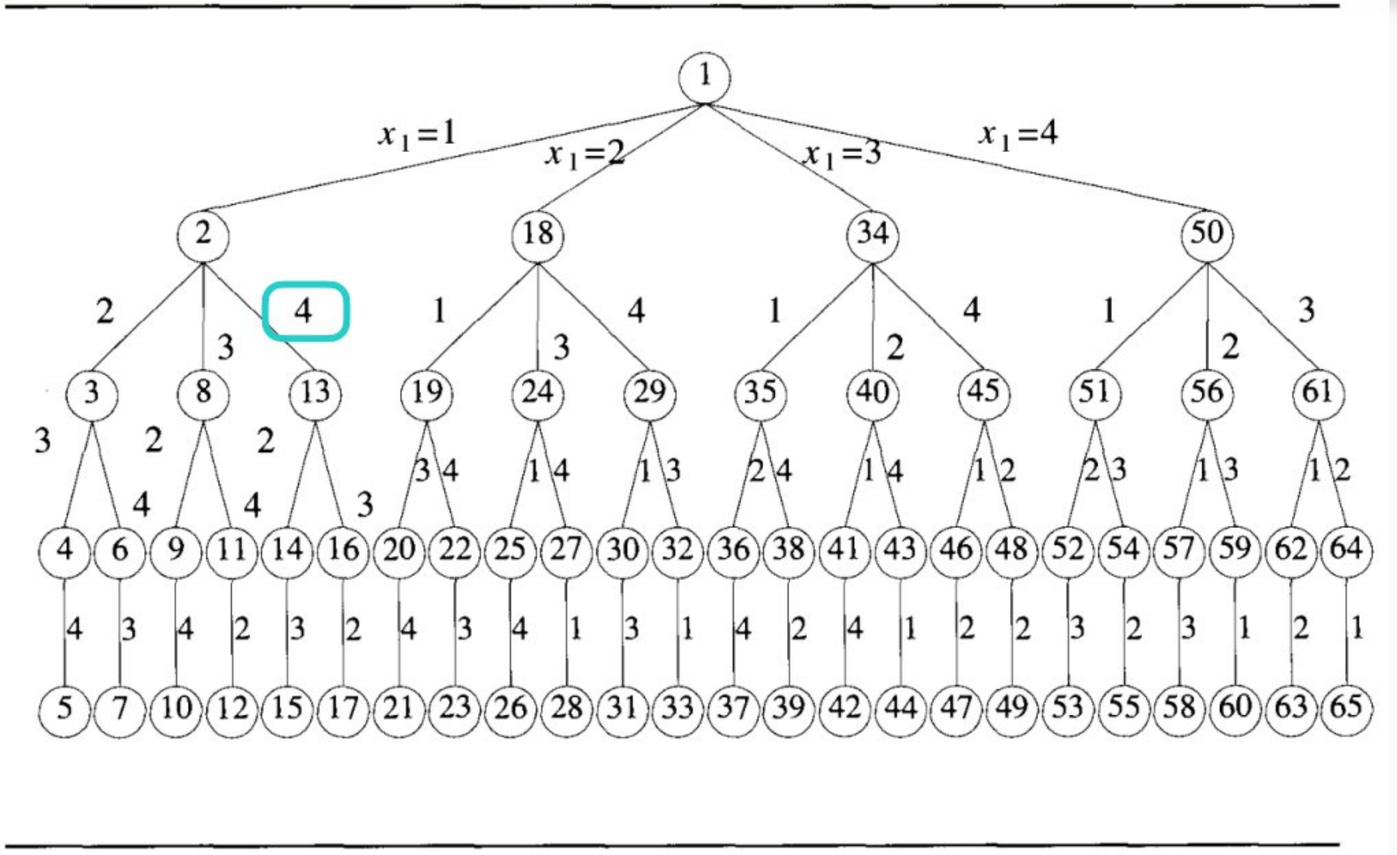
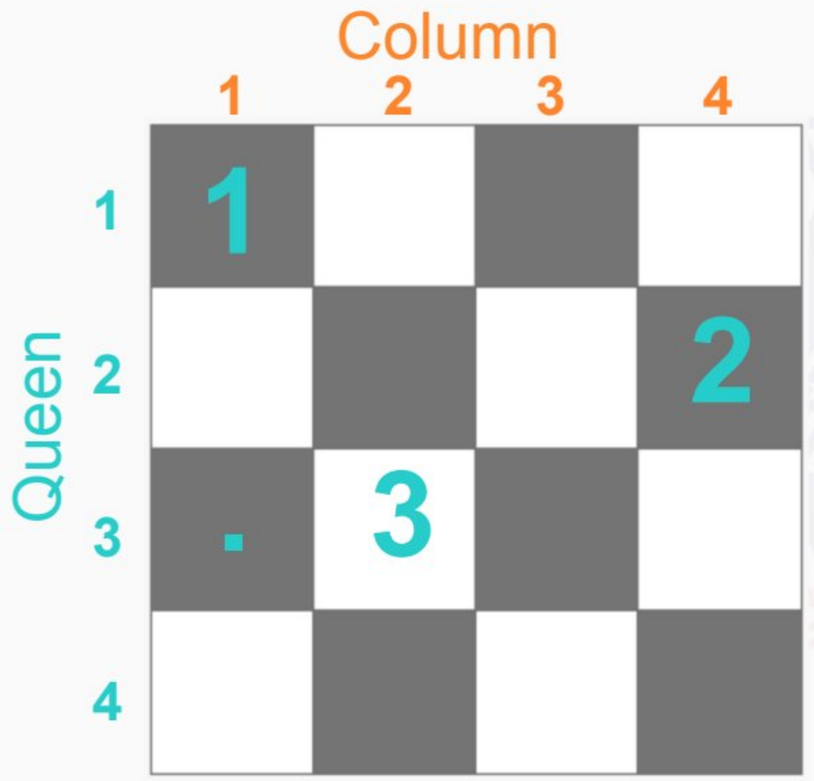


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



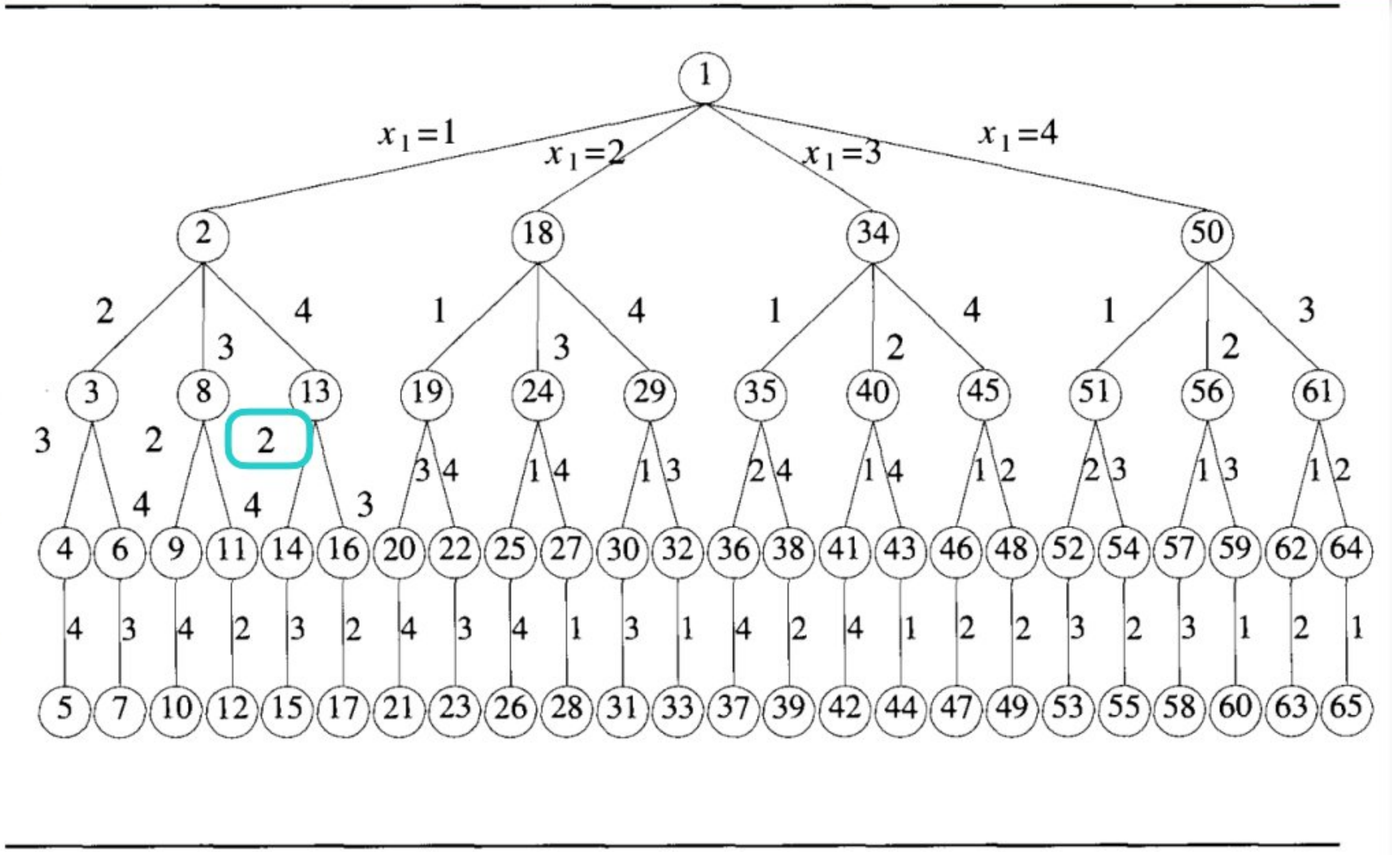
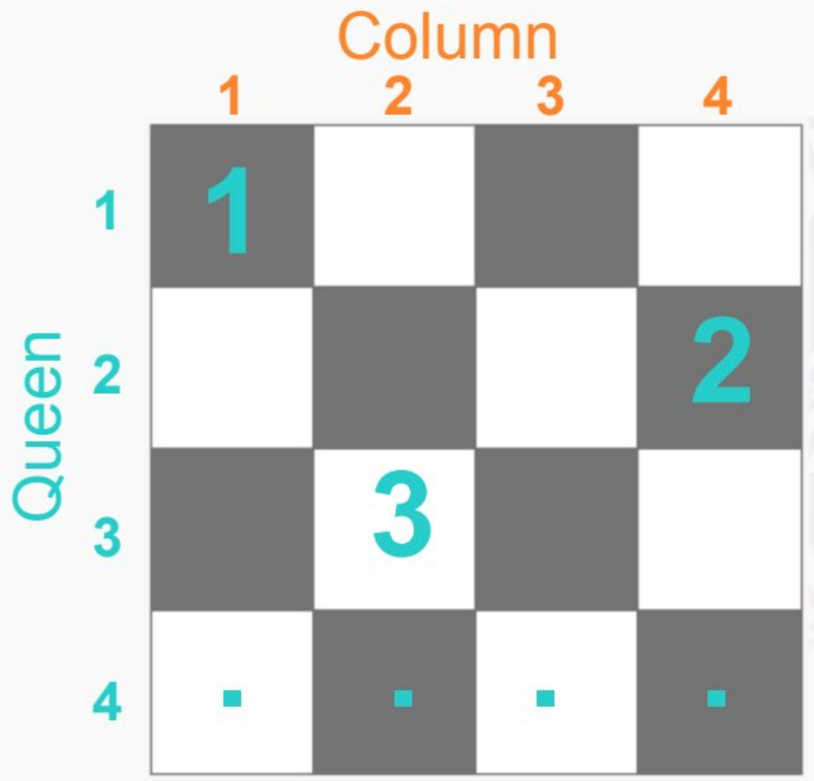


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



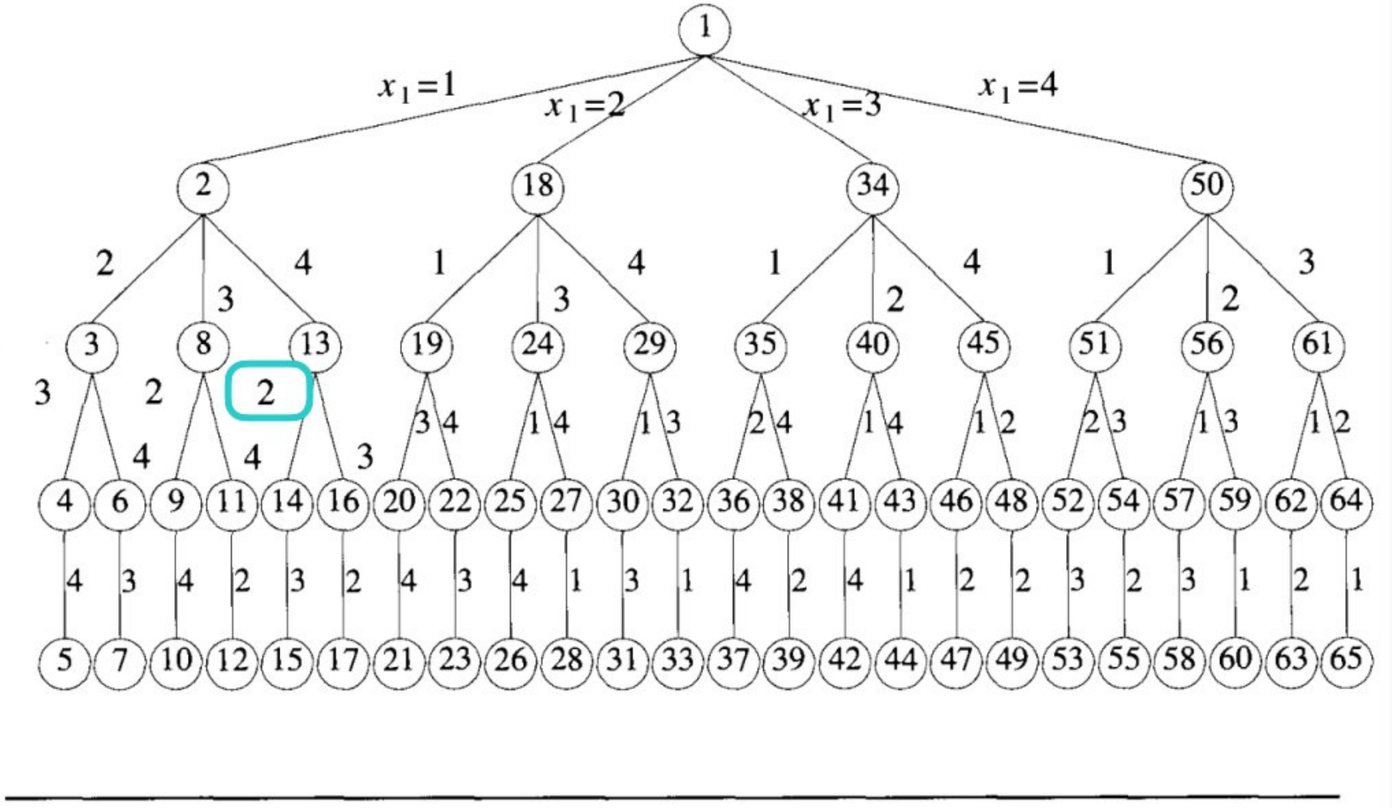
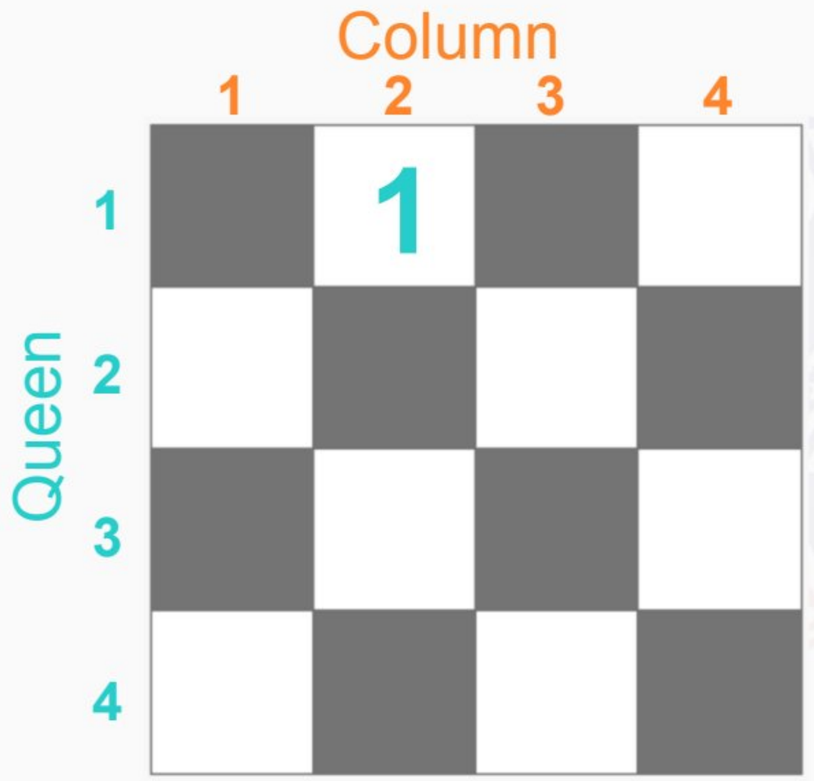


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



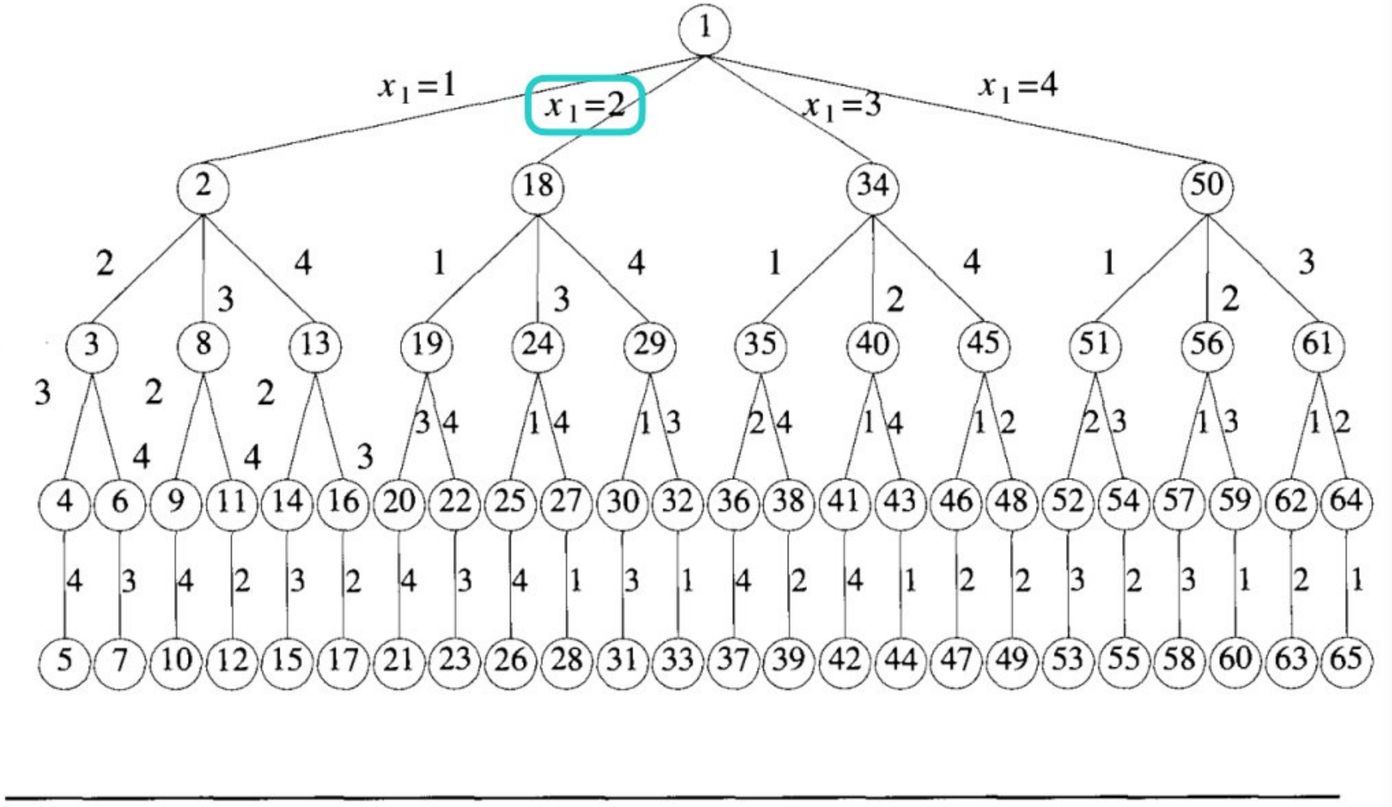
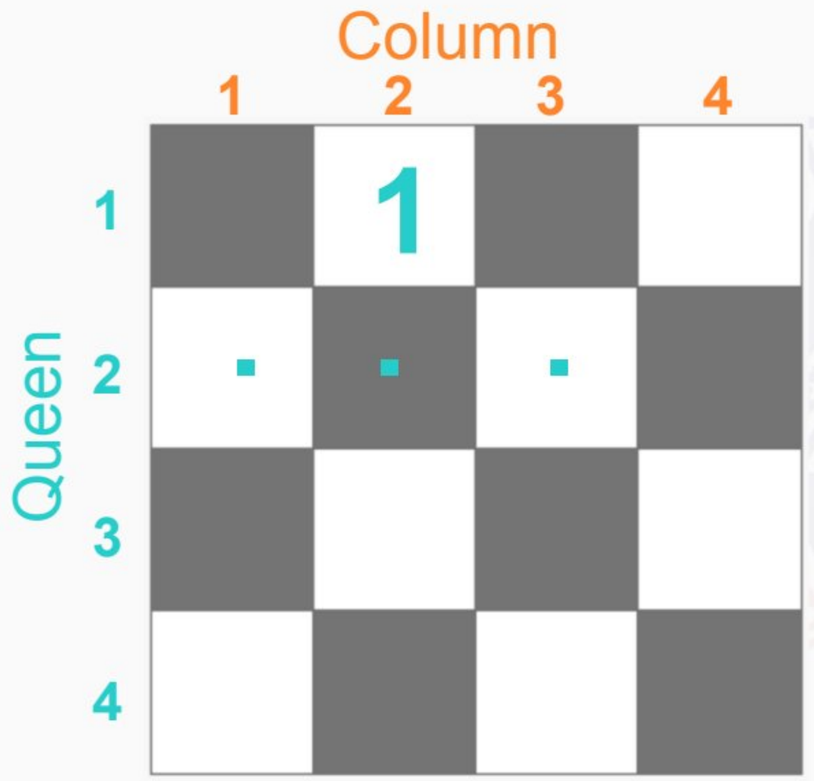


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



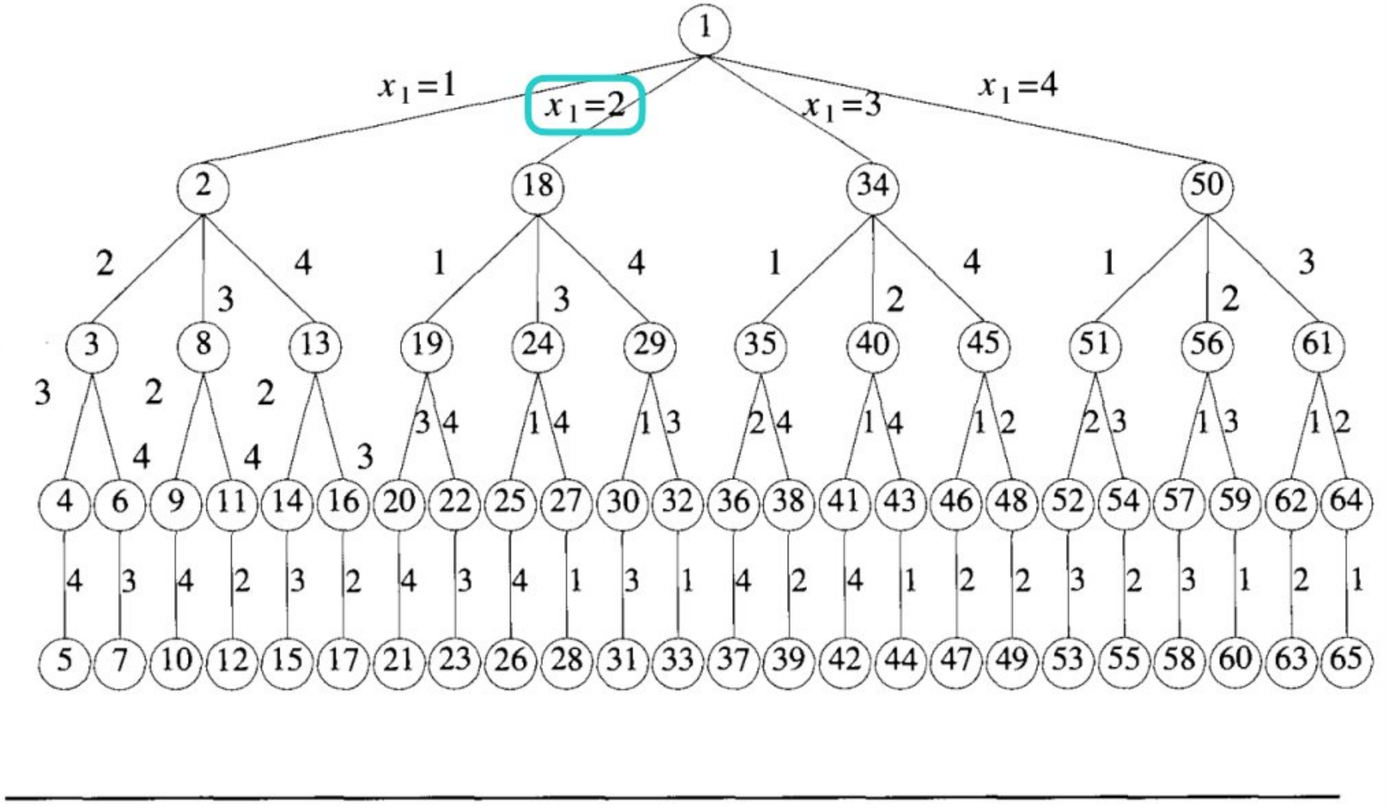
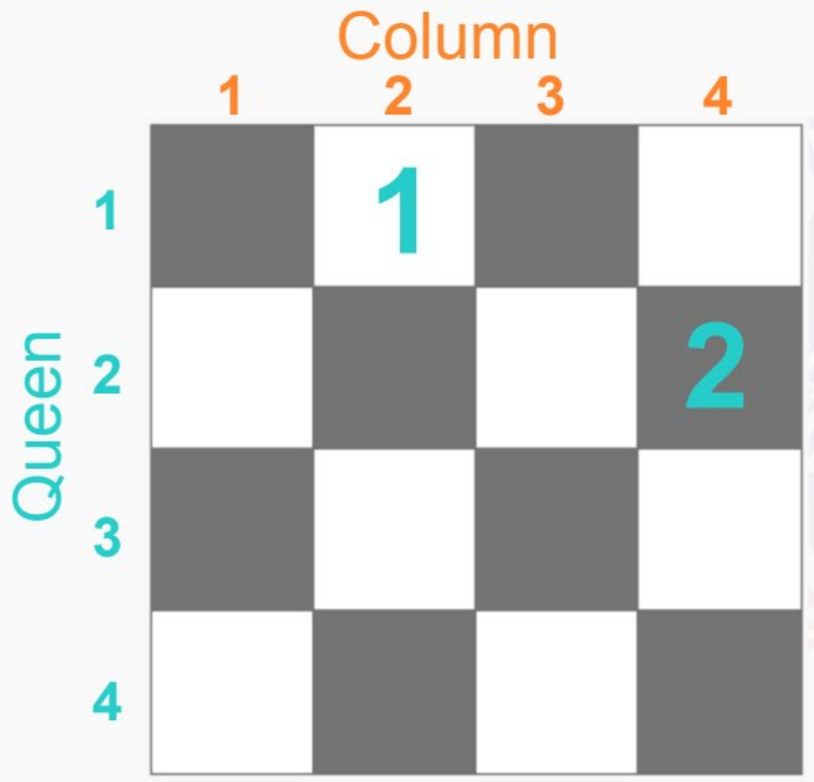


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



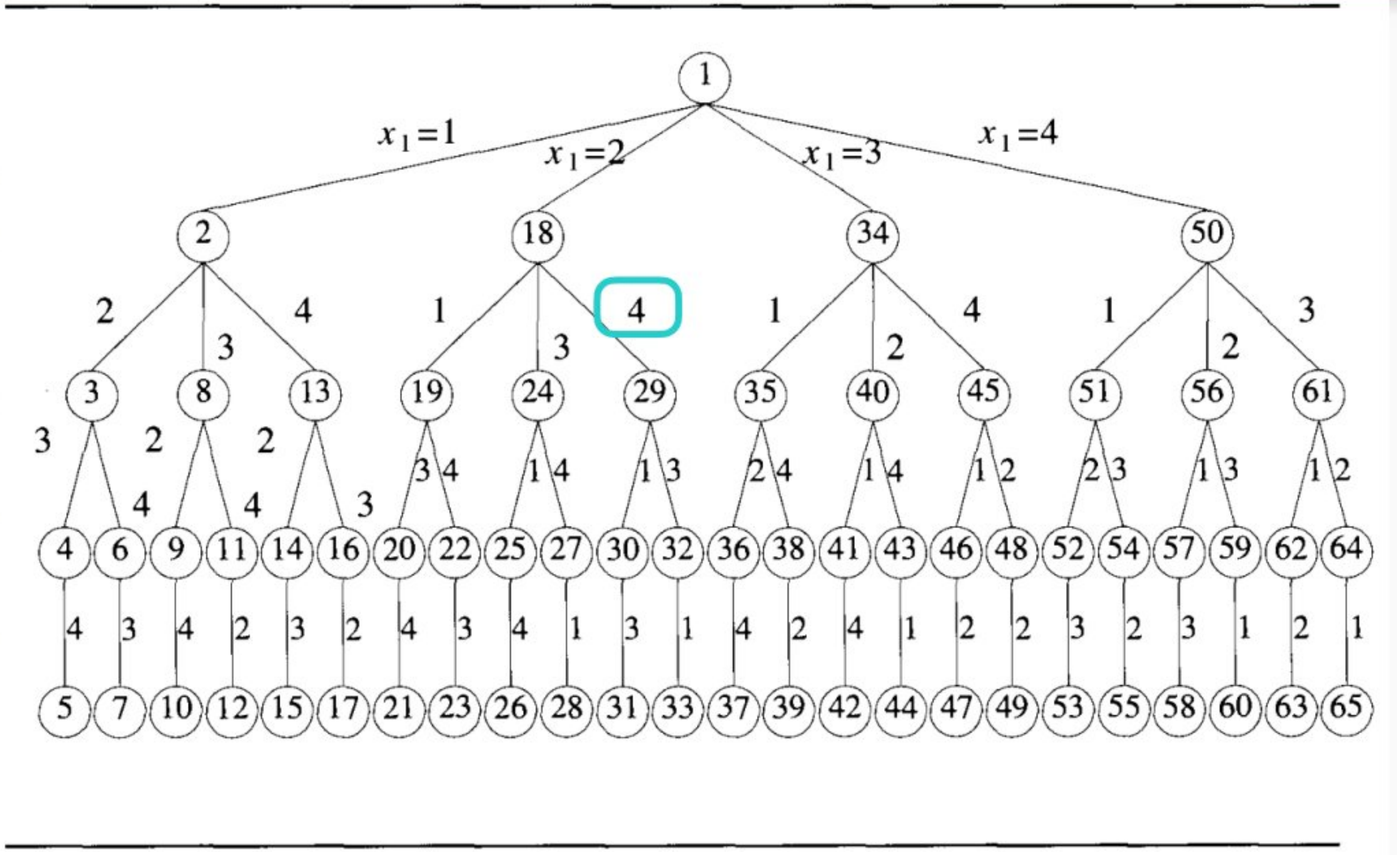
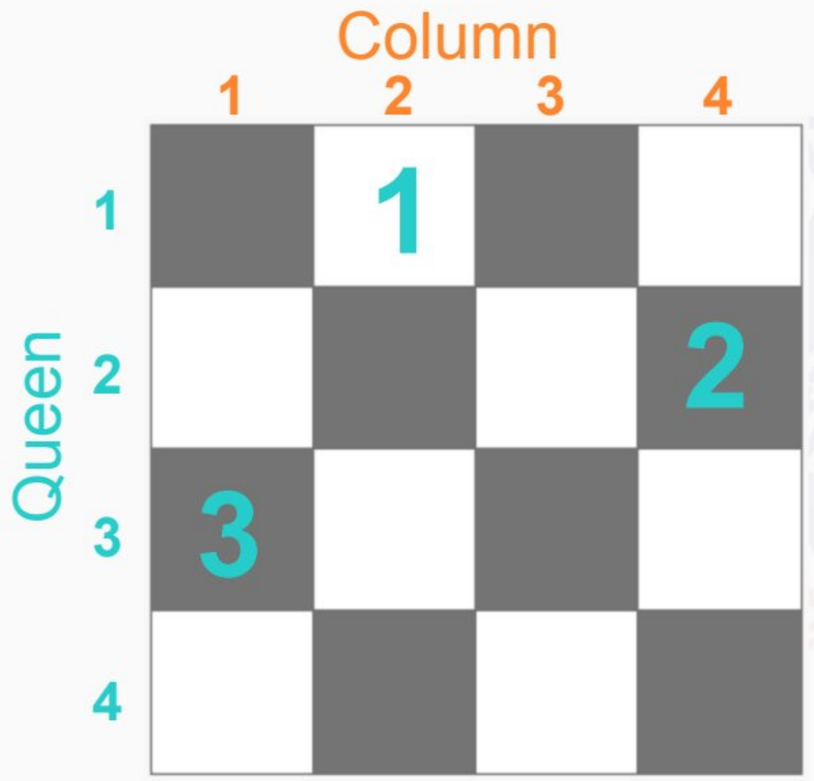


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



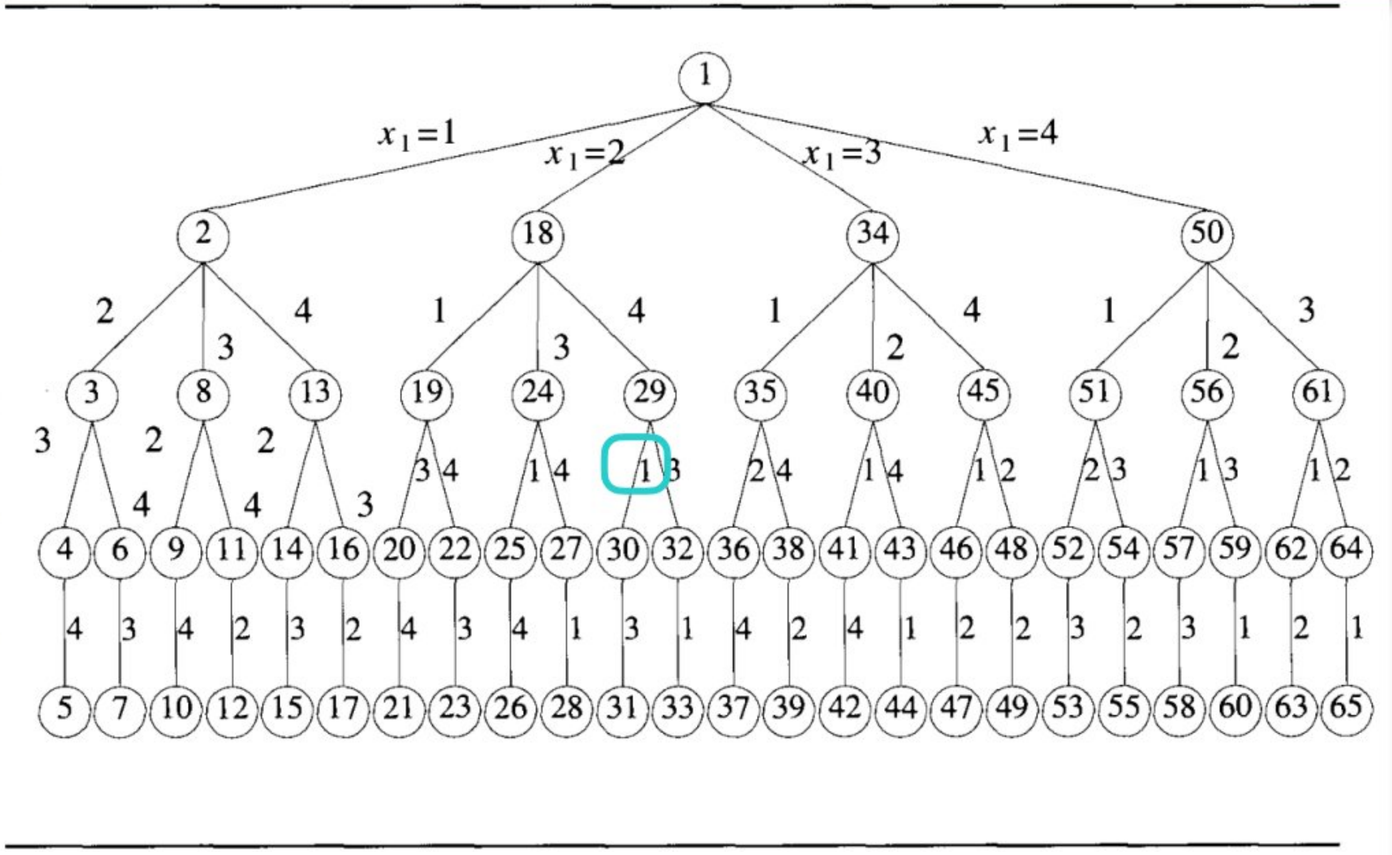
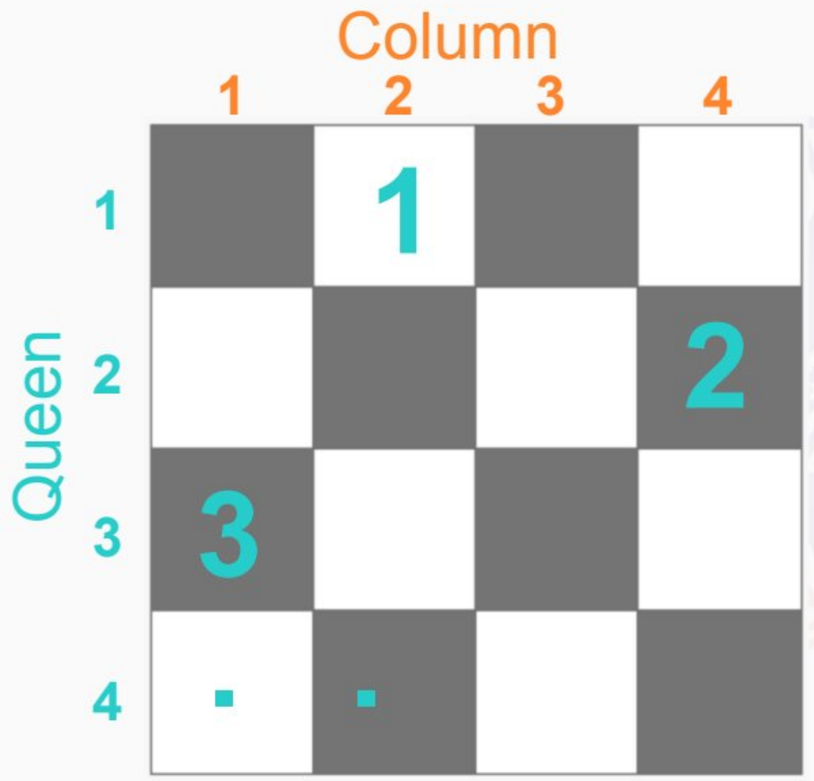


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



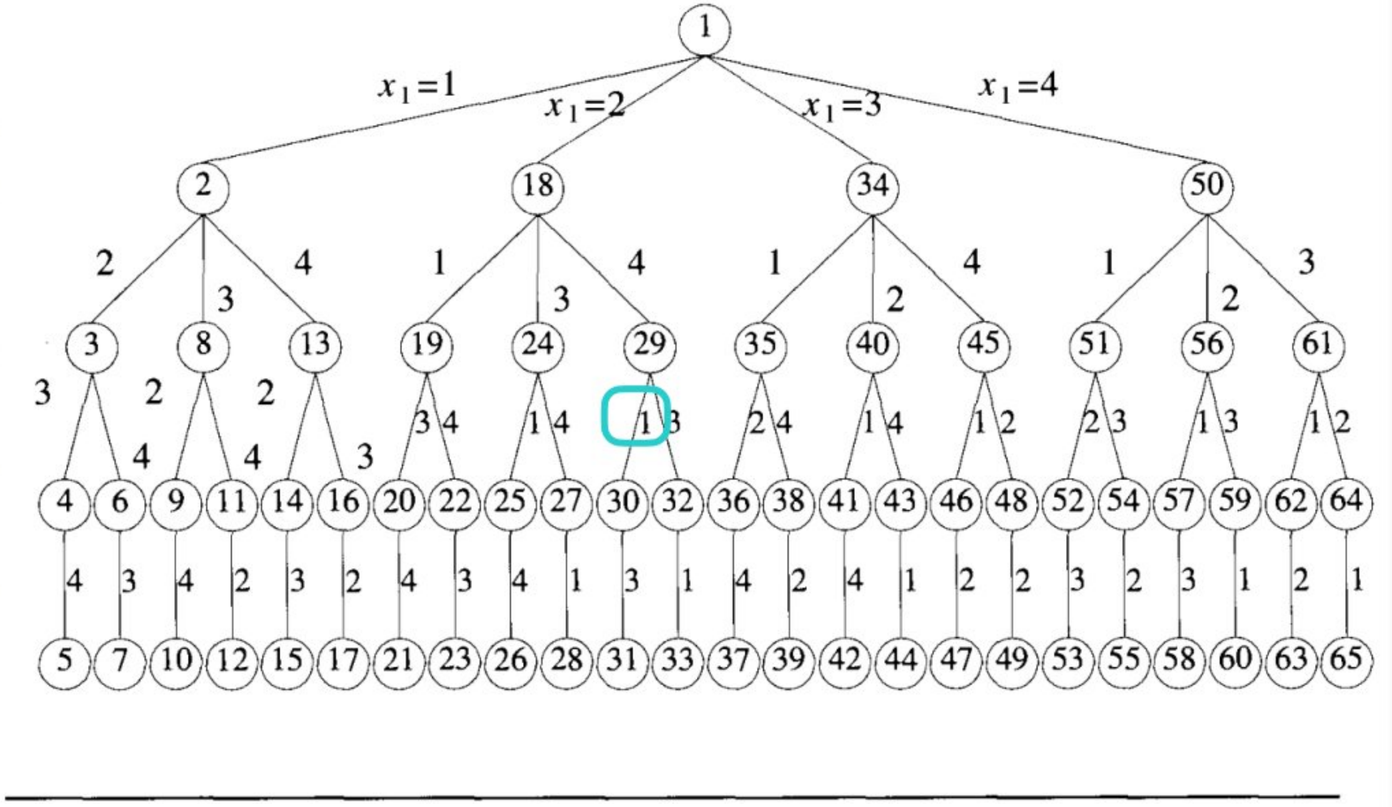
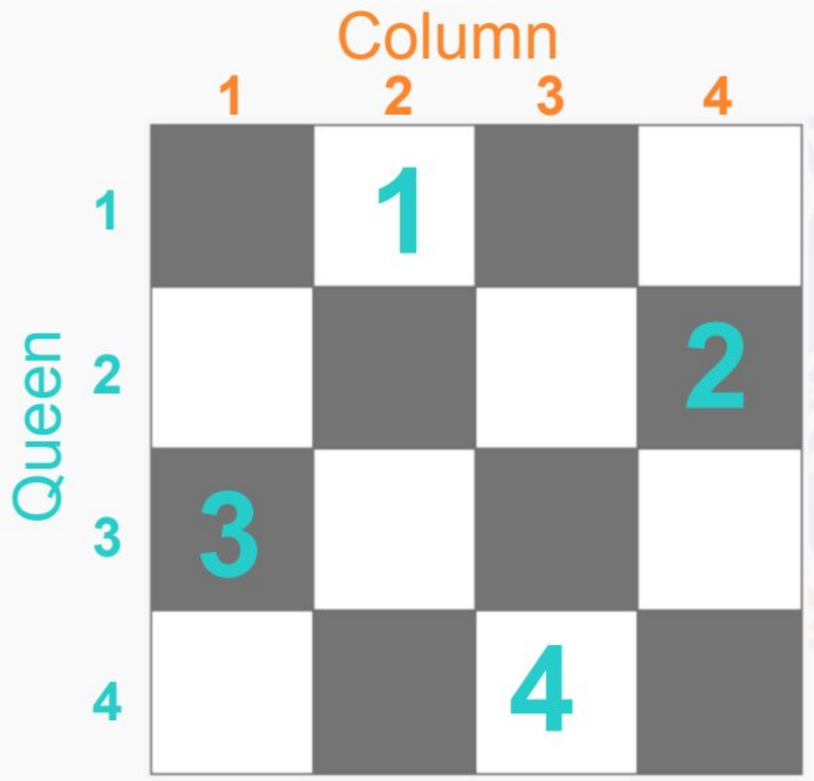


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



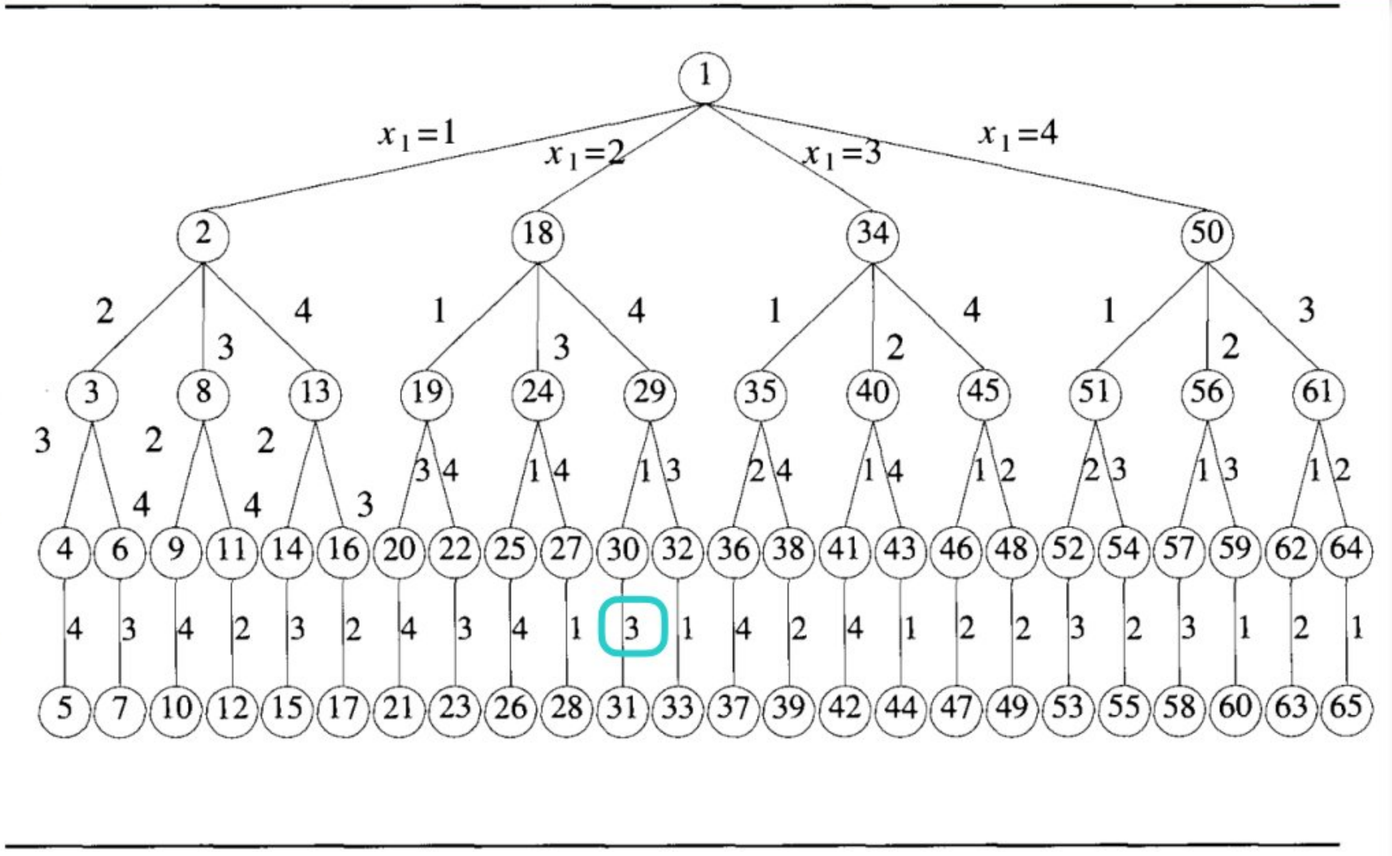
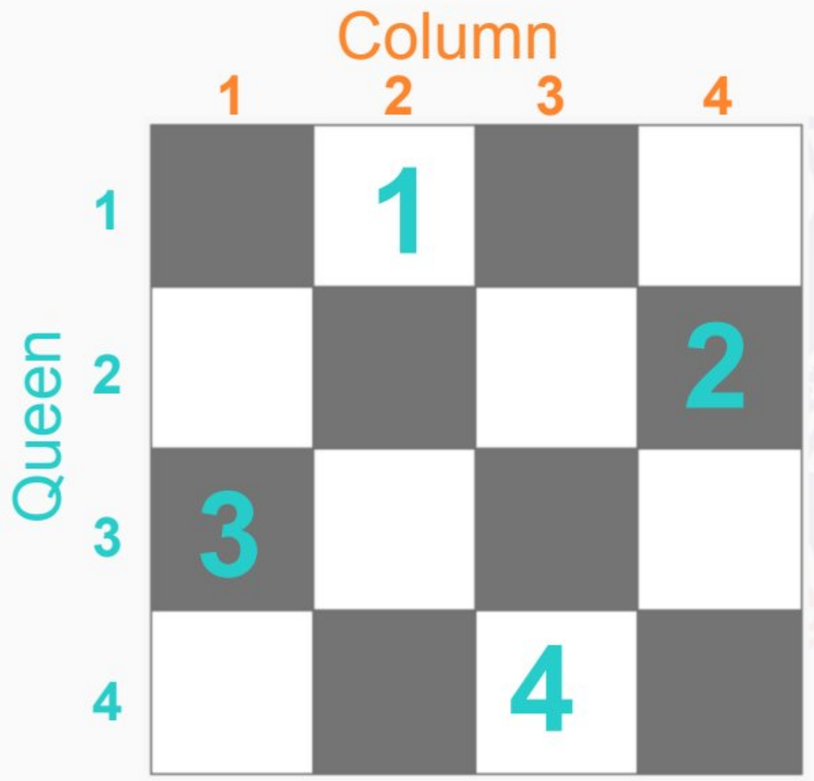


Figure 7.2 Tree organization of the 4-queens solution space. Nodes are numbered as in depth first search.

CamScanner

Backtracking Algorithm- 4 Queens Problem



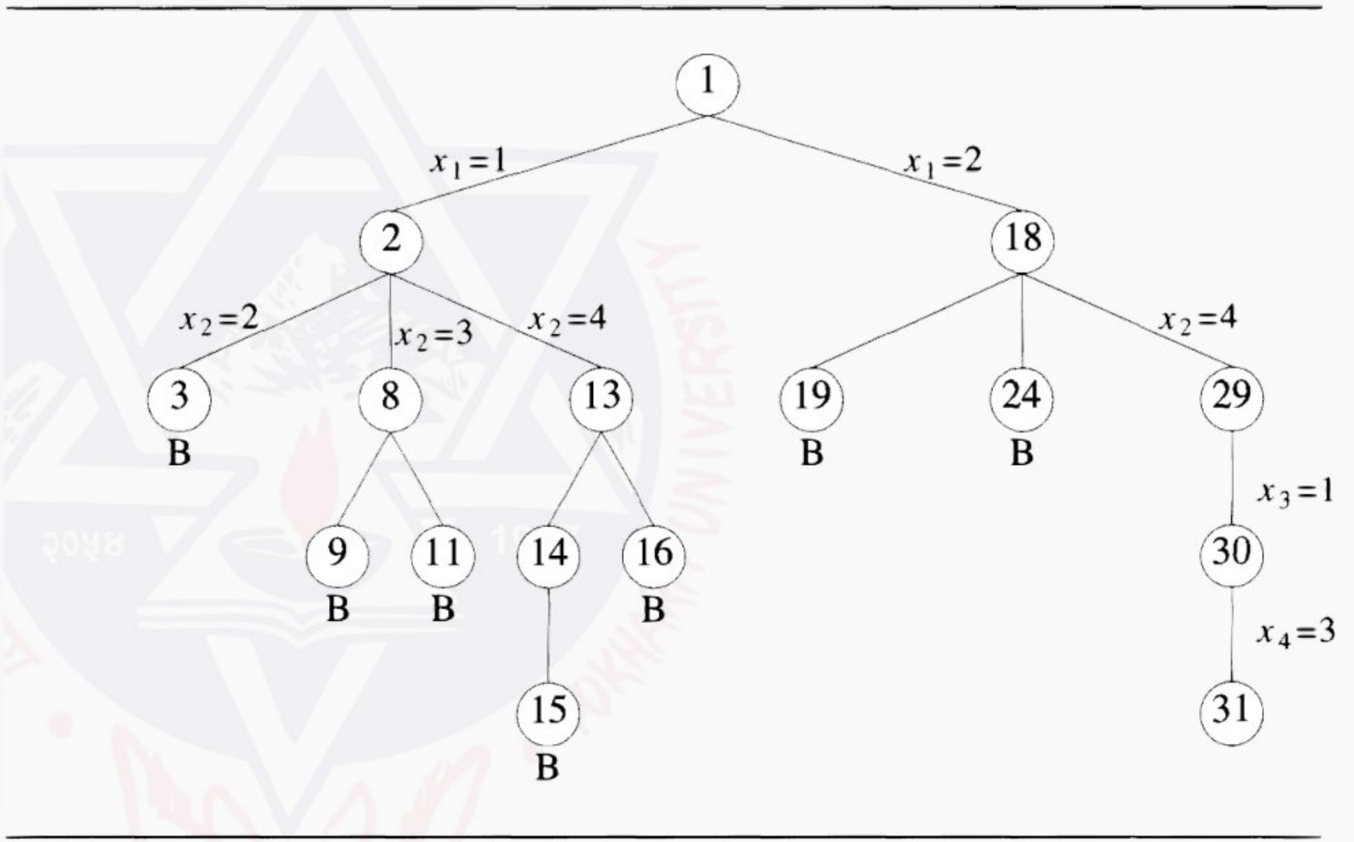


Figure 7.6 Portion of the tree of Figure 7.2 that is generated during backtracking

CamScanner

Backtracking Algorithm- 4 Queens Problem

- A node that gets killed as a result of the bounding function has a B under it.

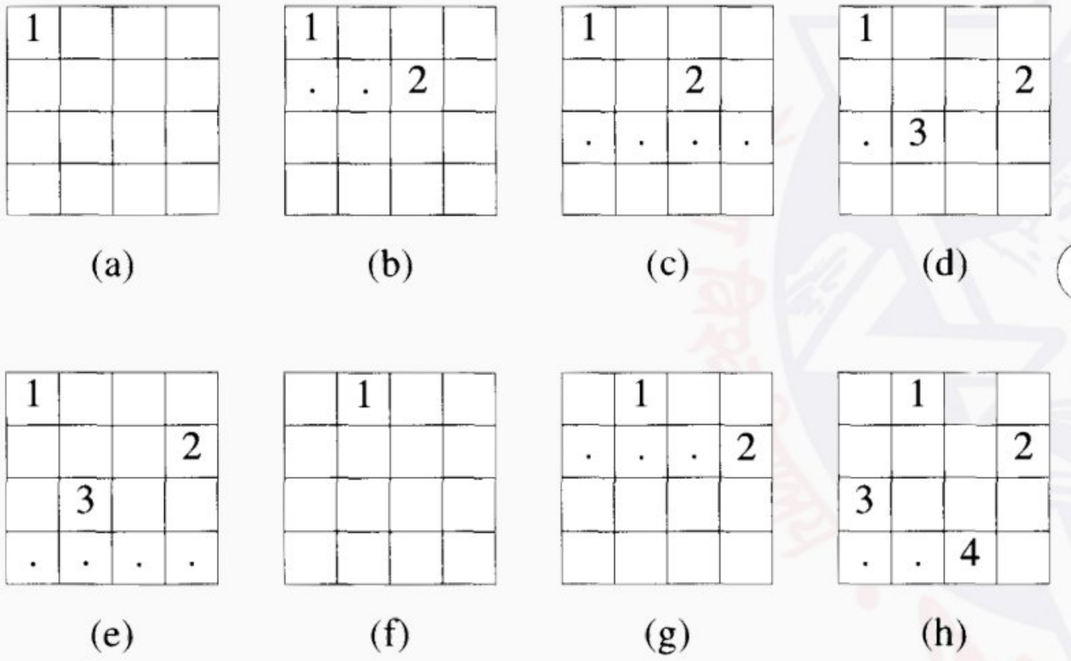


Figure 7.5 Example of a backtrack solution to the 4-queens problem

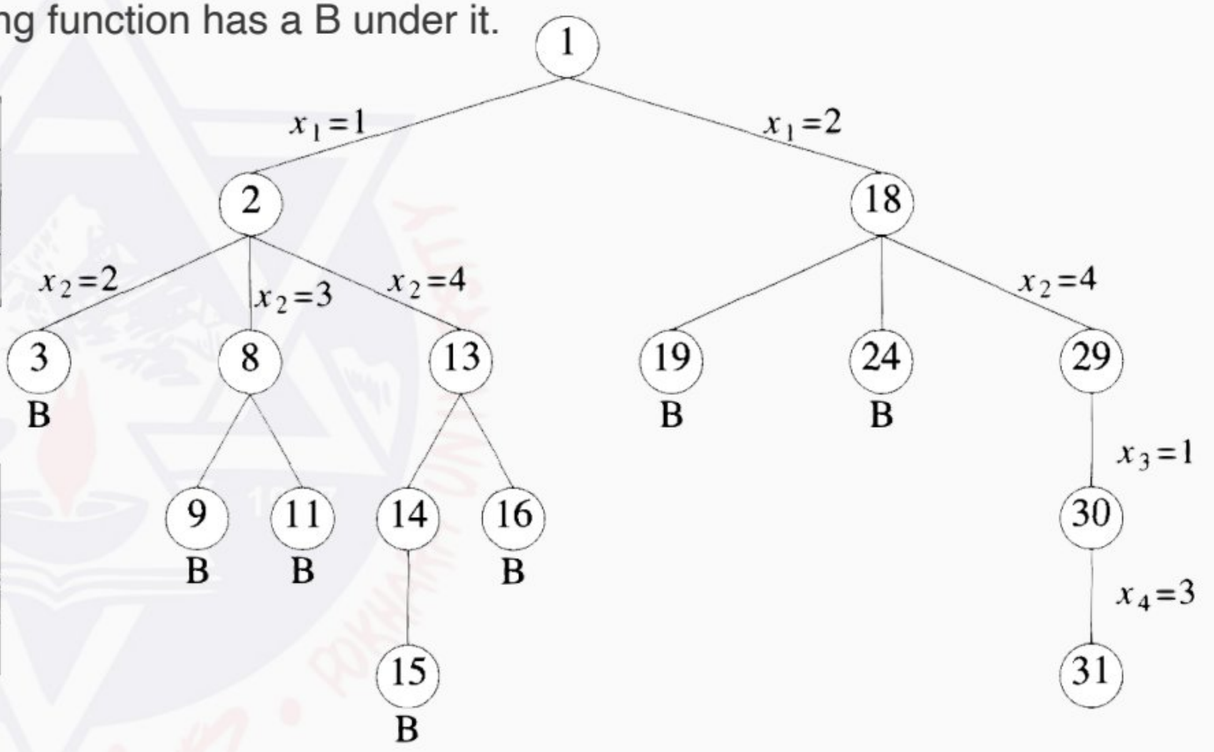


Figure 7.6 Portion of the tree of Figure 7.2 that is generated during backtracking

Backtracking Algorithm

```
1   Algorithm Backtrack(k)
2   // This schema describes the backtracking process using
3   // recursion. On entering, the first k − 1 values
4   // x[1], x[2], ..., x[k − 1] of the solution vector
5   // x[1 : n] have been assigned. x[ ] and n are global.
6   {
7       for (each x[k] ∈ T(x[1], ..., x[k − 1])) do
8       {
9           if (B_k(x[1], x[2], ..., x[k]) ≠ 0) then
10          {
11              if (x[1], x[2], ..., x[k] is a path to an answer node)
12                  then write (x[1 : k]);
13              if (k < n) then Backtrack(k + 1);
14          }
15      }
16  }
```

Algorithm 7.1 Recursive backtracking algorithm

CamScanner

Backtracking Algorithm

```
1   Algorithm IBacktrack(n)
2   // This schema describes the backtracking process.
3   // All solutions are generated in x[1 : n] and printed
4   // as soon as they are determined.
5   {
6       k := 1;
7       while (k ≠ 0) do
8       {
9           if (there remains an untried x[k] ∈ T(x[1], x[2], ...,
10              x[k − 1]) and B_k(x[1], ..., x[k]) is true) then
11          {
12                  if (x[1], ..., x[k] is a path to an answer node)
13                      then write (x[1 : k]);
14                  k := k + 1; // Consider the next set.
15          }
16          else k := k − 1; // Backtrack to the previous set.
17      }
18  }
```

Algorithm 7.2 General iterative backtracking method

CamScanner

Application of Backtracking

- **Constraint Satisfaction Problem**
  - N-Queen Problem
  - Graph Coloring
- **Combinatorial Optimization**
  - Generating Permutations and combinations
  - Traveling Salesman Problem
- **Puzzles and Games**
  - Maize Problems
  - Word Search and Crossword Puzzles



CamScanner

Assignment

- Branch and Bound Algorithms
- Randomized Algorithms



CamScanner



THANK YOU

CamScanner